



Slicer Developer Tutorial: Programming in Slicer

Sonia Pujol, Ph.D.

Assistant Professor of Radiology
Director of 3D Slicer Training & Education
Brigham and Women's Hospital
Harvard Medical School

Steve Pieper, Ph.D.

3D Slicer Chief Architect
Isomics Inc.

Goal of the tutorial



```
def threshold(t):  
    n=getNode('T2')  
    a=array('T2')  
    a[a<t]=0  
    arrayFromVolumeModified(n)  
    print('Thresholding done')
```



```
b=qt.QPushButton('Toggle')  
b.connect('clicked()',toggle)  
b.styleSheet = "font-size: 24pt; color:  
aqua; margin: 20px"  
b.show()
```

This tutorial is an introduction to the Python interactor and the Qt widget toolkit in 3D Slicer release version 5

Tutorial Outline



Part 1: 3D Slicer Modules Overview



Part 2: Getting Familiar with the Python environment in 3D Slicer

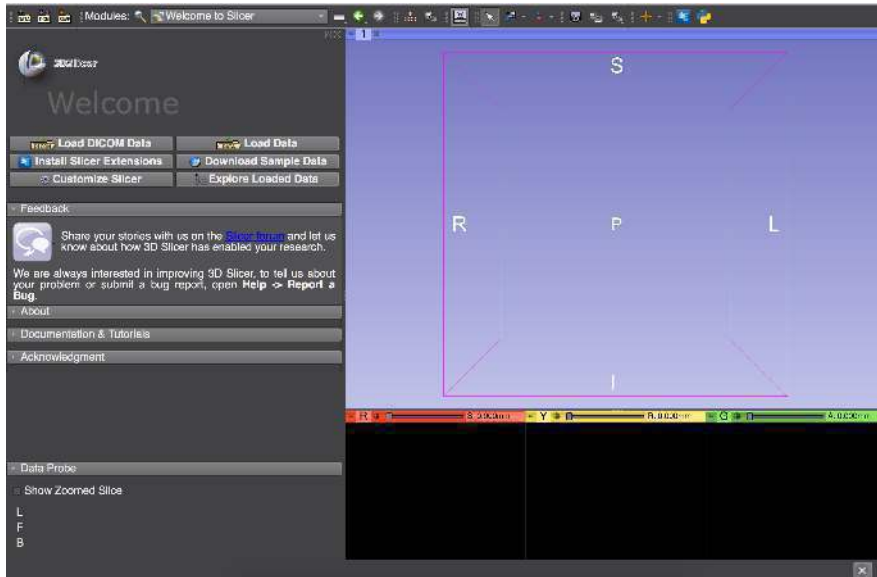


Part 3: Getting Familiar with the Qt widget toolkit in 3D Slicer

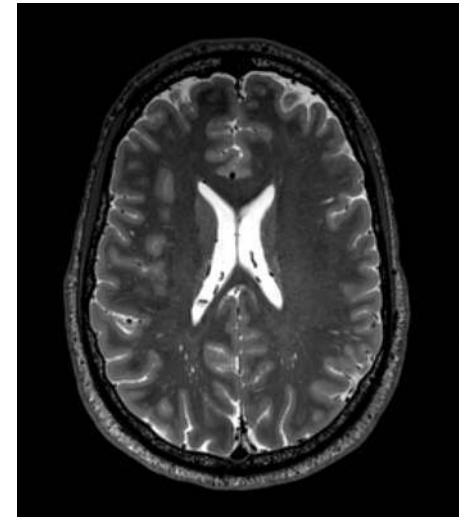
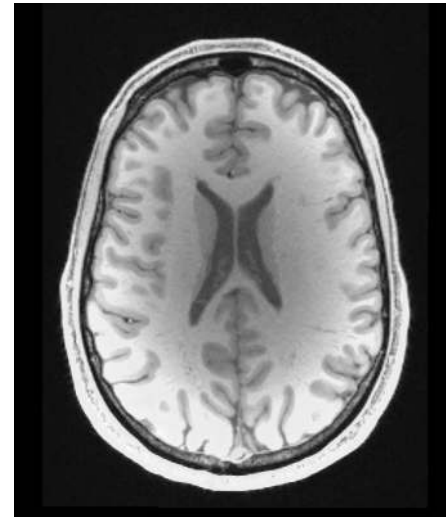
Disclaimer

- 3D Slicer is a free open source software application distributed under a BSD style license.
- The software is not FDA approved or CE-Marked, and is for research use only.

Tutorial materials



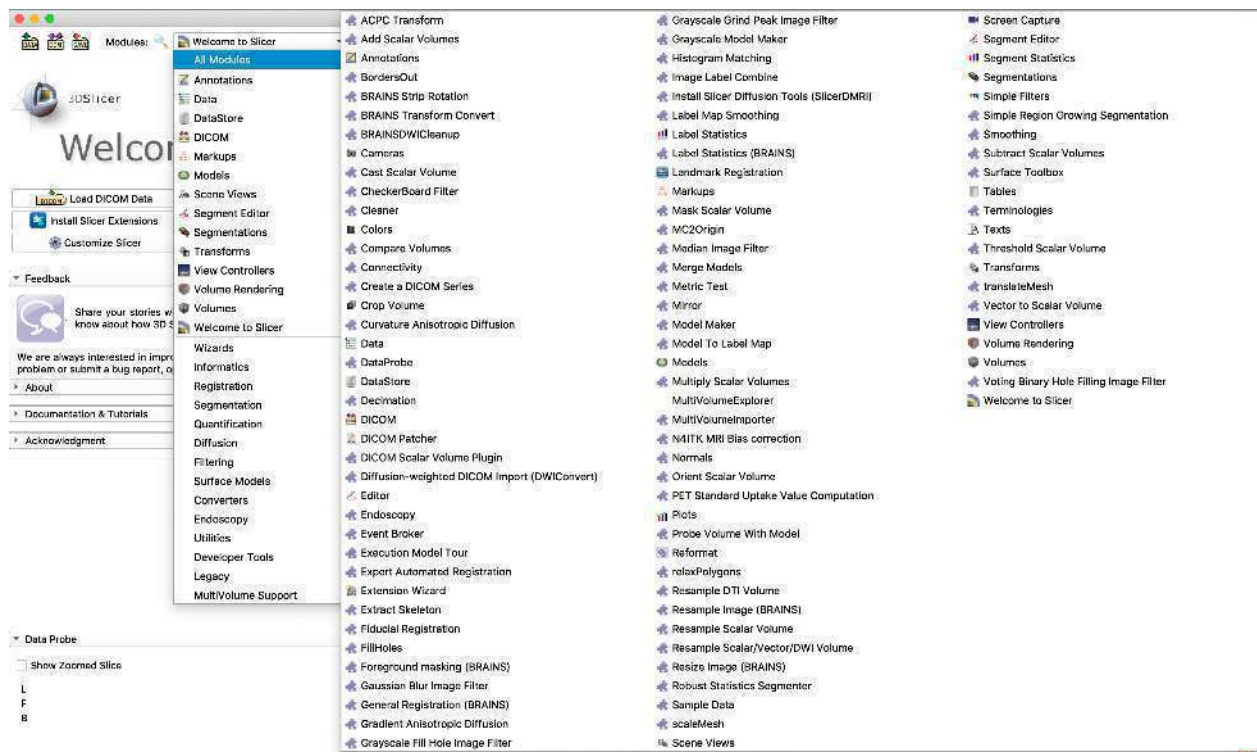
3D Slicer version 4.11



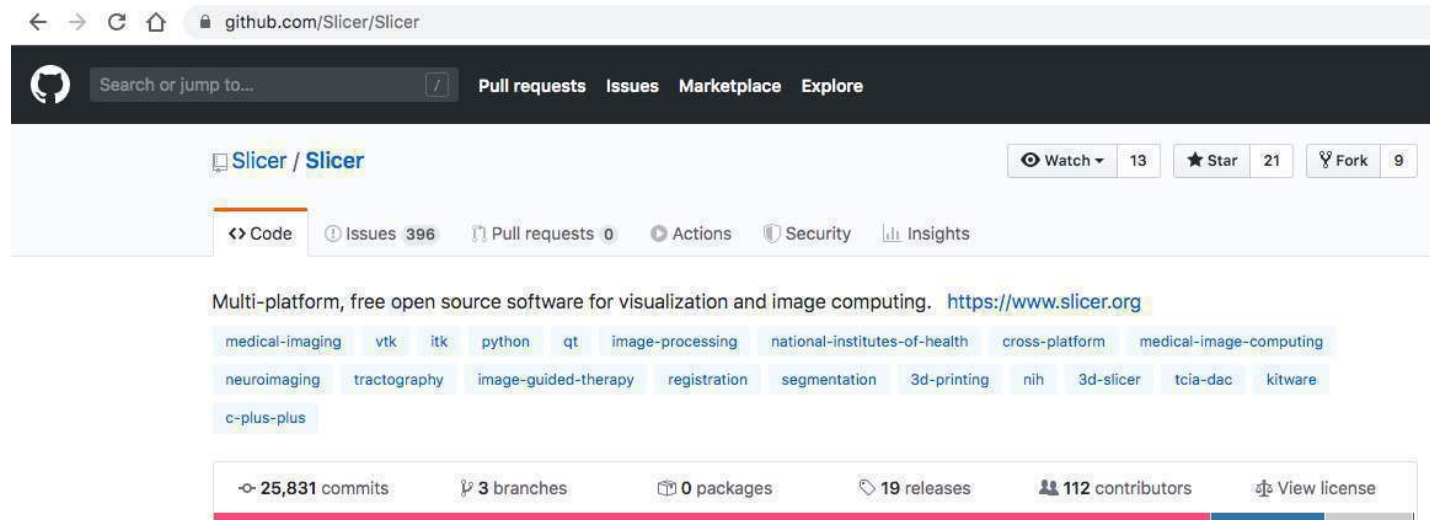
SlicerProgrammingTutorialData.zip

Part 1

Slicer Modules Overview



3D Slicer



- 3D Slicer is an open-source platform for the analysis and visualization of medical imaging data
- 3D Slicer is compiled and tested every day on Windows, Mac, and Linux platforms
- The source code is freely available on GitHub at <http://github.com/Slicer/Slicer>

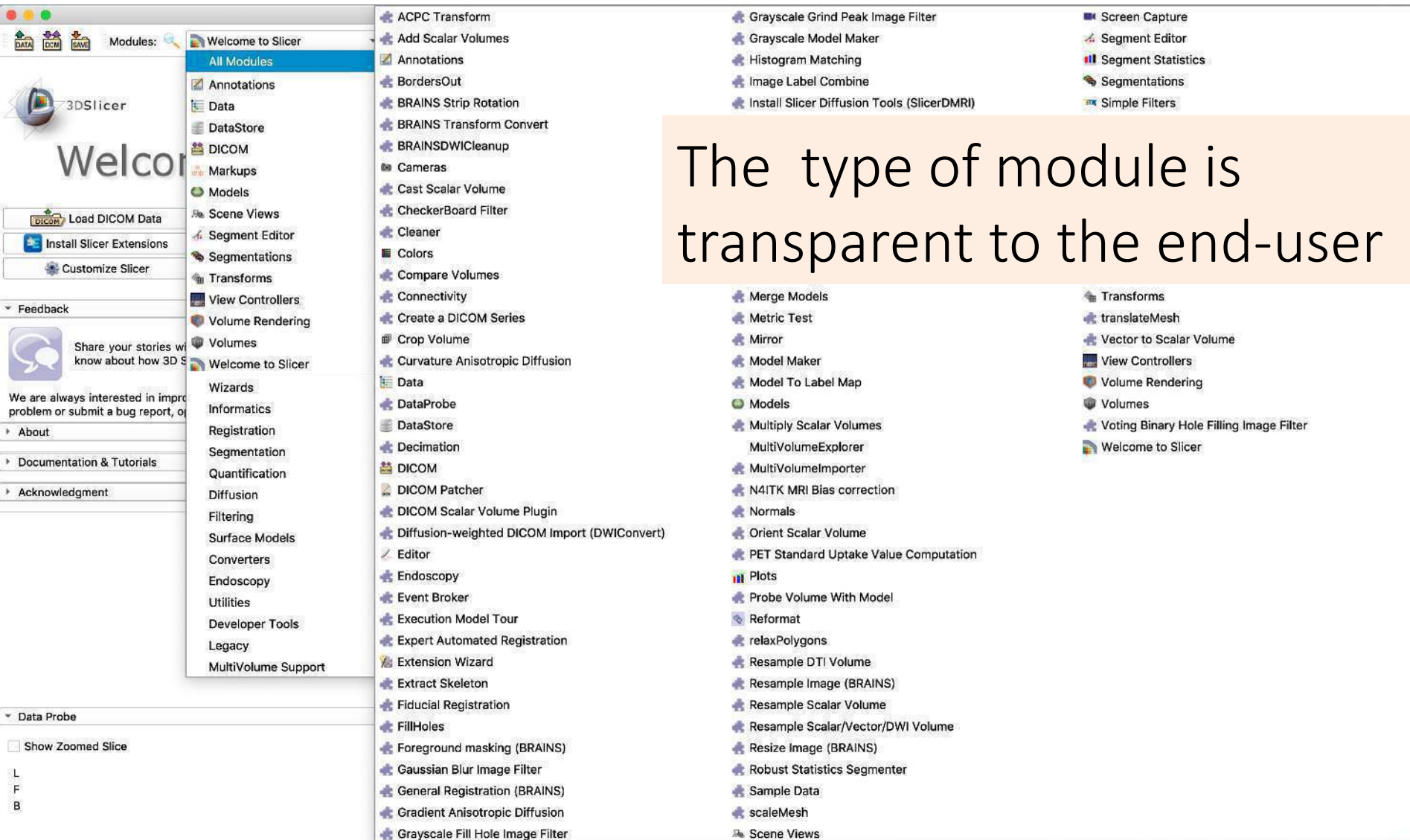
Slicer Modules

3D Slicer supports three types of modules:

- **Command Line Interface (CLI):** standalone executable with limited input/output arguments
- **Loadable Modules (C++ Plugins):** optimized for heavy computation
- **Scripted Modules (Python):** recommended for fast prototyping and workflow development

Focus of this tutorial

Slicer Modules

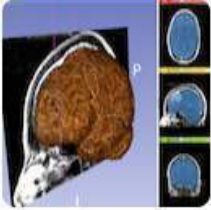
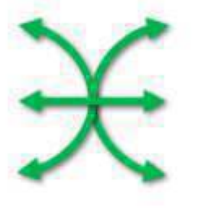
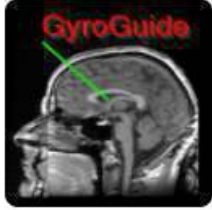
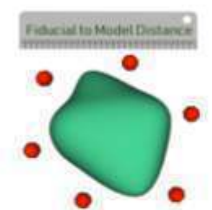
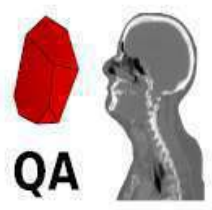


The screenshot displays the Slicer application interface. On the left, the 'Modules' menu is open, showing a list of categories and sub-modules. The 'All Modules' option is selected. The main window shows a list of available modules, including 'ACPC Transform', 'Add Scalar Volumes', 'Annotations', 'BordersOut', 'BRAINS Strip Rotation', 'BRAINS Transform Convert', 'BRAINSDWICleanup', 'Cameras', 'Cast Scalar Volume', 'CheckerBoard Filter', 'Cleaner', 'Colors', 'Compare Volumes', 'Connectivity', 'Create a DICOM Series', 'Crop Volume', 'Curvature Anisotropic Diffusion', 'Data', 'DataProbe', 'DataStore', 'Decimation', 'DICOM', 'DICOM Patcher', 'DICOM Scalar Volume Plugin', 'Diffusion-weighted DICOM Import (DWIConvert)', 'Editor', 'Endoscopy', 'Event Broker', 'Execution Model Tour', 'Expert Automated Registration', 'Extension Wizard', 'Extract Skeleton', 'Fiducial Registration', 'FillHoles', 'Foreground masking (BRAINS)', 'Gaussian Blur Image Filter', 'General Registration (BRAINS)', 'Gradient Anisotropic Diffusion', 'Grayscale Fill Hole Image Filter', 'Grayscale Grind Peak Image Filter', 'Grayscale Model Maker', 'Histogram Matching', 'Image Label Combine', 'Install Slicer Diffusion Tools (SlicerDMRI)', 'Merge Models', 'Metric Test', 'Mirror', 'Model Maker', 'Model To Label Map', 'Models', 'Multiply Scalar Volumes', 'MultiVolumeExplorer', 'MultiVolumeImporter', 'N4ITK MRI Bias correction', 'Normals', 'Orient Scalar Volume', 'PET Standard Uptake Value Computation', 'Plots', 'Probe Volume With Model', 'Reformat', 'relaxPolygons', 'Resample DTI Volume', 'Resample Image (BRAINS)', 'Resample Scalar Volume', 'Resample Scalar/Vector/DWI Volume', 'Resize Image (BRAINS)', 'Robust Statistics Segmenter', 'Sample Data', 'scaleMesh', 'Scene Views', 'Screen Capture', 'Segment Editor', 'Segment Statistics', 'Segmentations', 'Simple Filters', 'Transforms', 'translateMesh', 'Vector to Scalar Volume', 'View Controllers', 'Volume Rendering', 'Volumes', 'Voting Binary Hole Filling Image Filter', and 'Welcome to Slicer'.

The type of module is transparent to the end-user

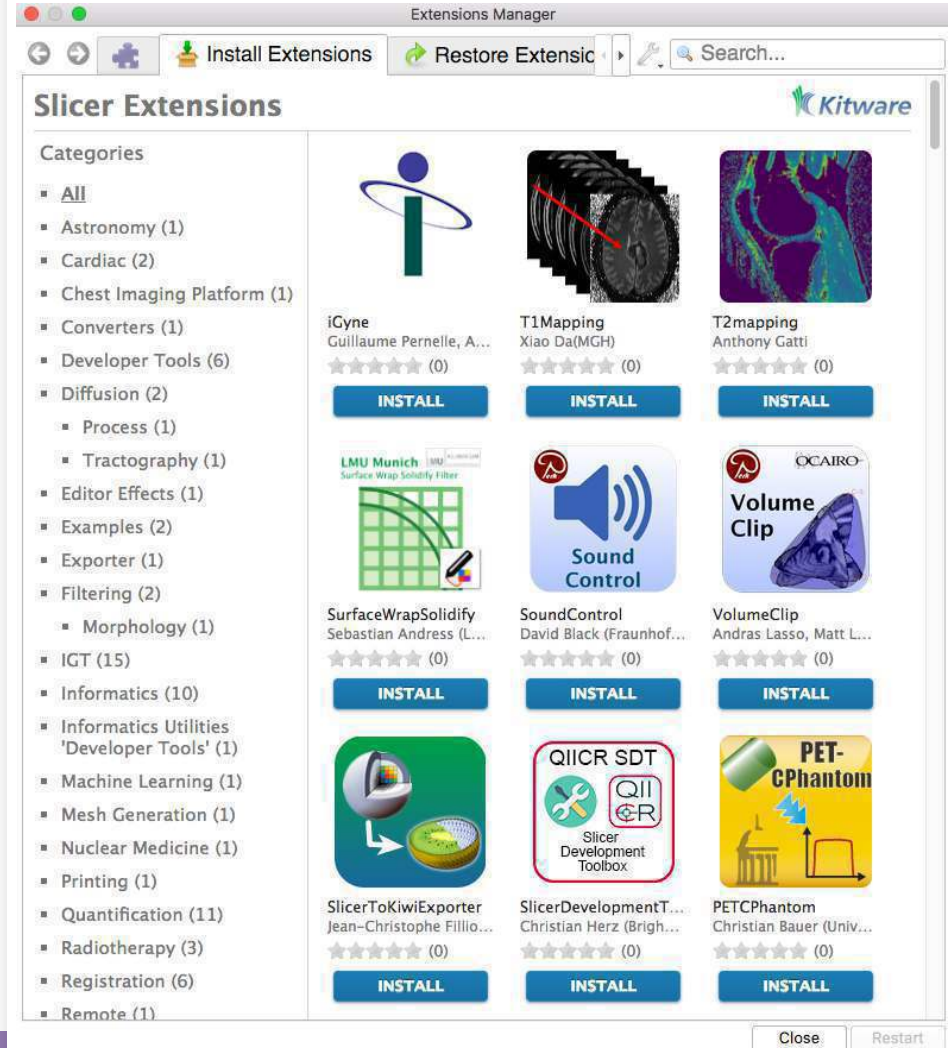
Slicer Extensions

A Slicer Extension is a delivery package bundling together one or more Slicer modules

			
SwissSkullStripper Bill Lorensen (Noware...) ★★★★★ (0)	PETTumorSegmenta... Christian Bauer (Univ...) ★★★★★ (0)	SlicerOpenIGTLink Junichi Tokuda (SPL), ... ★★★★★ (0)	GyroGuide Ruifeng Chen, Luping... ★★★★★ (0)
INSTALL	INSTALL	INSTALL	INSTALL
			
PET-IndiC Ethan Ulrich (Universi...) ★★★★★ (0)	FiducialsToModelDi... Jesse Reynolds (Cante...) ★★★★★ (0)	Slicer-Wasp Thomas Lawson (MR...) ★★★★★ (0)	ImageCompare Paolo Zaffino (Magna ...) ★★★★★ (0)
INSTALL	INSTALL	INSTALL	INSTALL

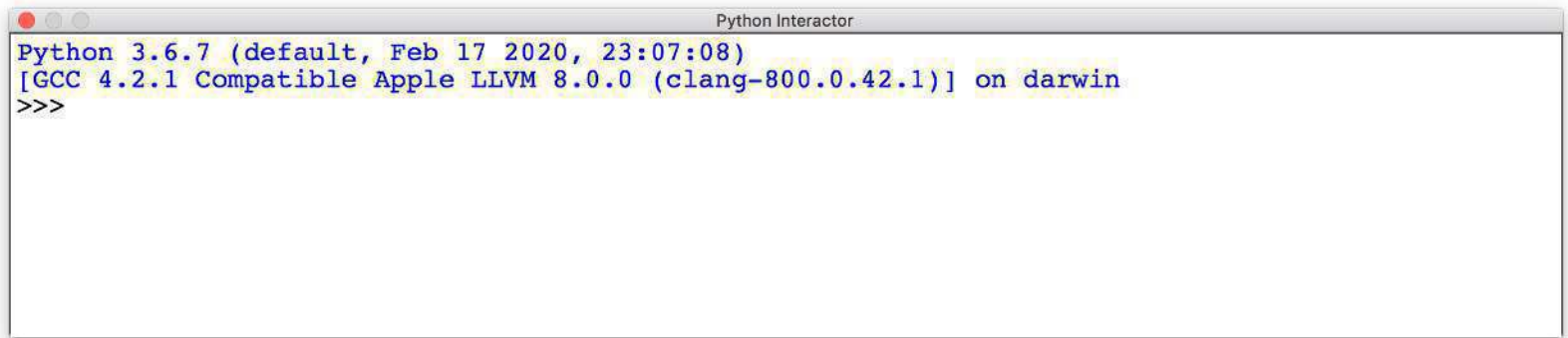
Slicer Extension Manager

- The Slicer Extension Manager provides an 'App store' platform for the 3D Slicer ecosystem
- The Extension Manager enables an easy creation and installation of Slicer extensions
- Slicer release version 5 includes over 130 extensions



Part 2

Getting Familiar with the Python environment in 3D Slicer



```
Python 3.6.7 (default, Feb 17 2020, 23:07:08)
[GCC 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)] on darwin
>>>
```

Python in Slicer

Slicer v.4.11 works with **Python3** and a rich set of standard libraries



NumPy is the fundamental package for scientific computing with Python.



VTK is an open-source library for manipulating and displaying scientific data.



ITK is an open-source library for image analysis.

CTK is an open-source library for biomedical image computing.



PythonQt is a Python binding for Qt.



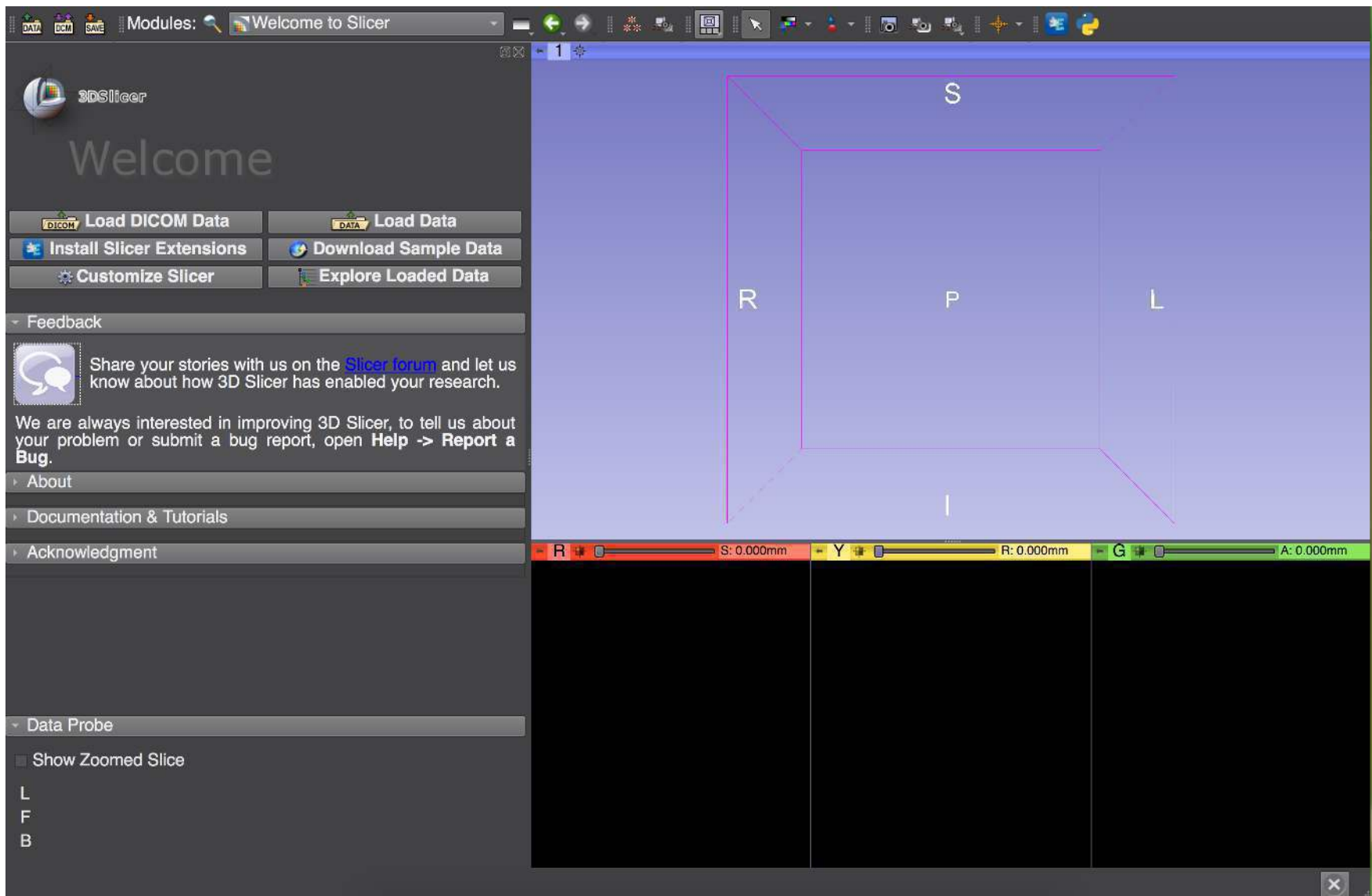
Qt is a cross-platform framework used as a graphical toolkit.

Python in Slicer



The **Python Package index (PyPi)** gives access to over 200,000 additional Python packages (<http://pipy.org>)

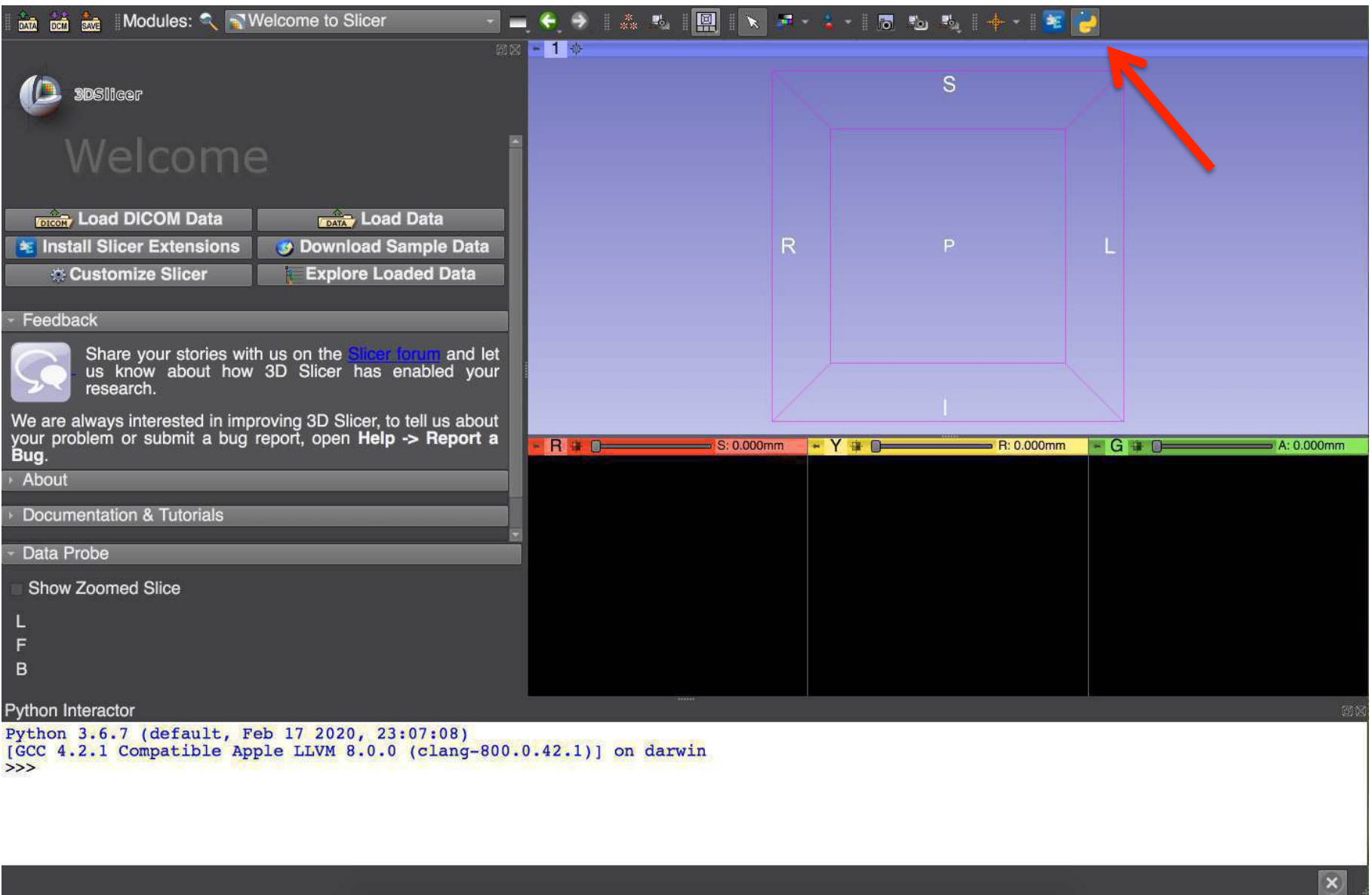
- The **pip install** command in Slicer enables developers to install most common scientific computing tools (e.g. TensorFlow, SciPy, PyTorch, Pandas, etc.)
- Slicer can be used as a **Jupyter notebook** kernel
- PyCharm and other Python development tools can be used with Slicer



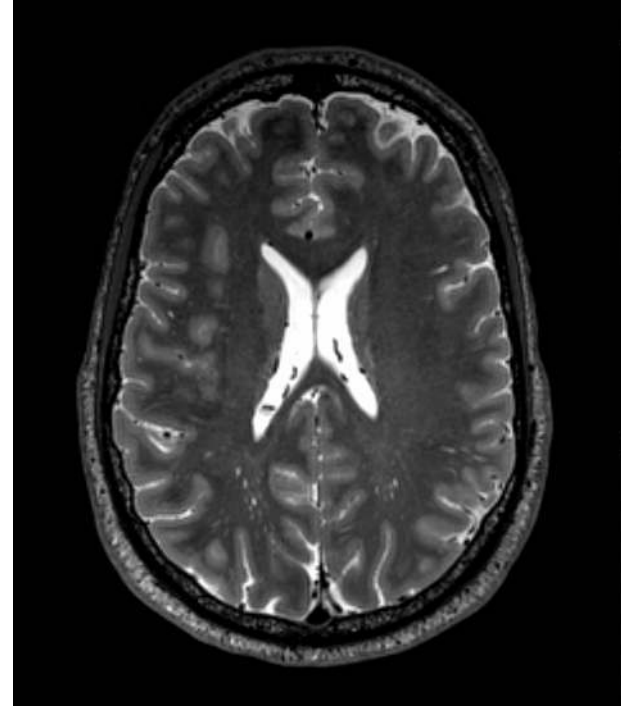
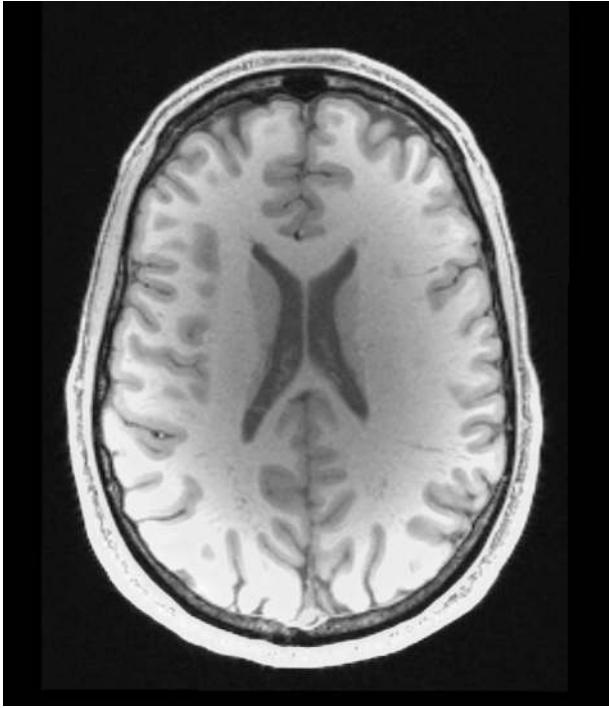
Slicer release version 5 integrates
Python3, VTK5 and ITK5

The Python Console in Slicer

The Python Interactor is a Qt-based console that enables direct access to Slicer MRML Nodes, libraries (NumPy, VTK, ITK, CTK) and Qt.

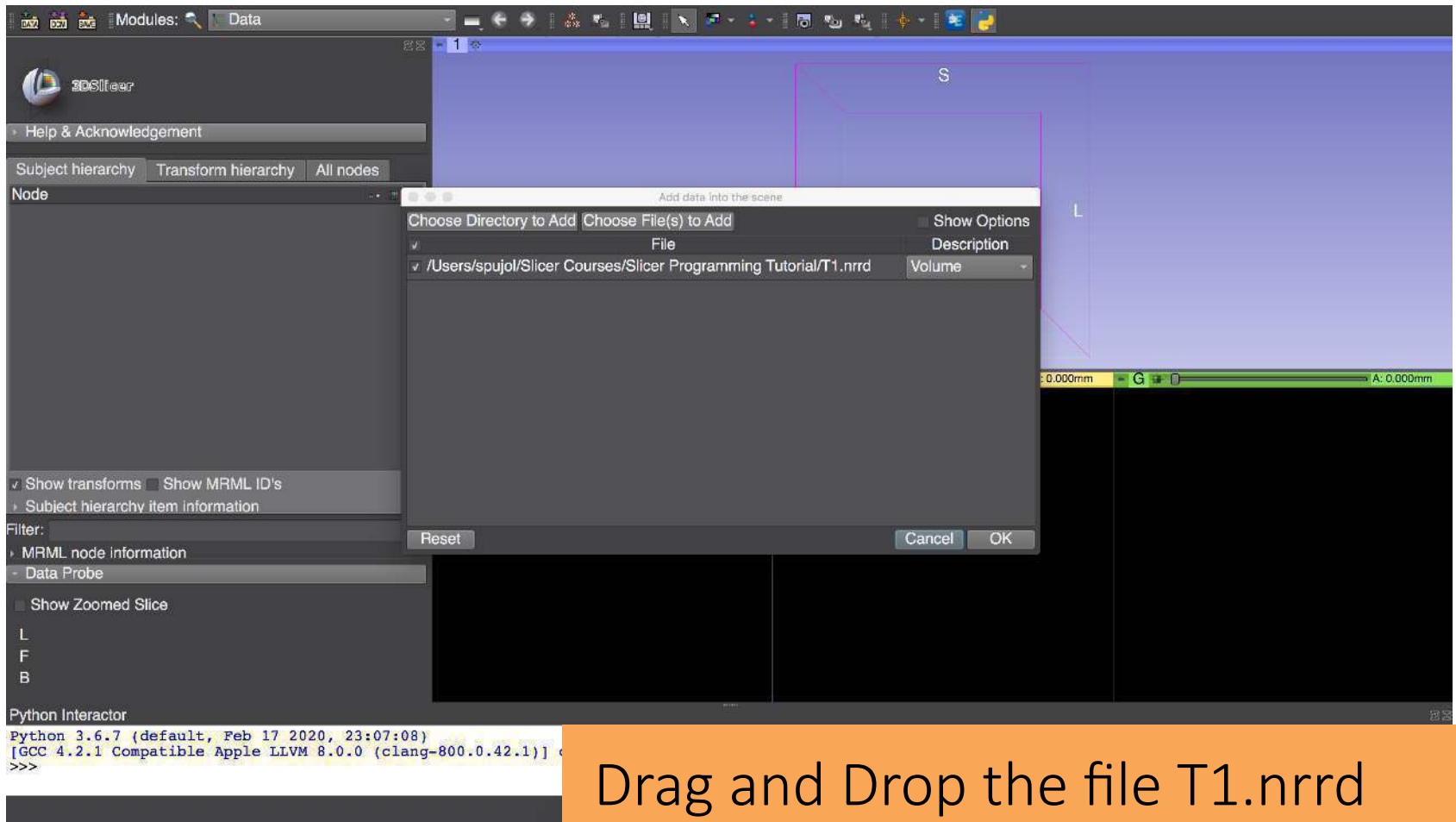


To access the Python Interactor, click on the Python icon  in the top bar menu of Slicer.



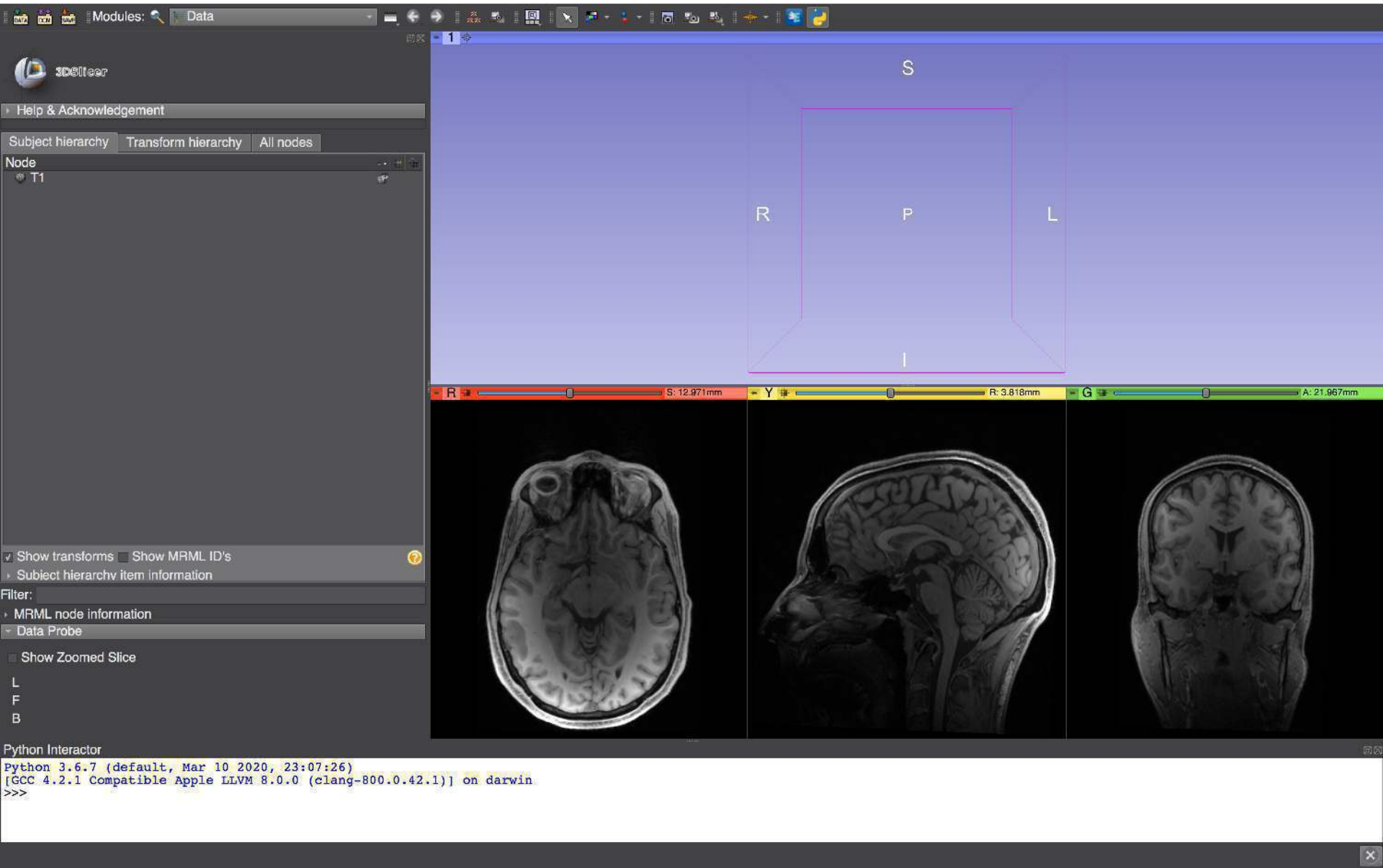
The Slicer Programming tutorial dataset includes a T1-weighted and a T2-weighted MRI scan of a healthy subject

Tutorial dataset



Drag and Drop the file T1.nrrd
Click on OK to load the file in Slicer

Tutorial dataset



Big Picture

- Slicer is free and open-source software
- There are thousands of sophisticated medical images available on the Internet that you could visualize and analyze with 3D Slicer

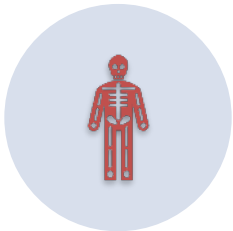
Slicer Data Model



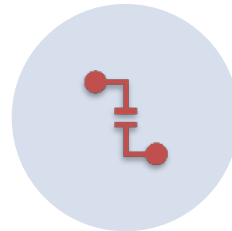
The **Slicer Data Model** is based on the Slicer Scene Data Structure



A **Slicer scene** is a collection of images, annotations, 3D models, spatial transforms, fiducials and cameras



The **Medical Reality Markup Language (MRML)** is an XML-based language used to serialize the content of Slicer scene on disk (scene.mrml)



Each element a scene is called a **MRML node**

Slicer MRML Nodes: Basic Types



Data Node: Stores the raw data

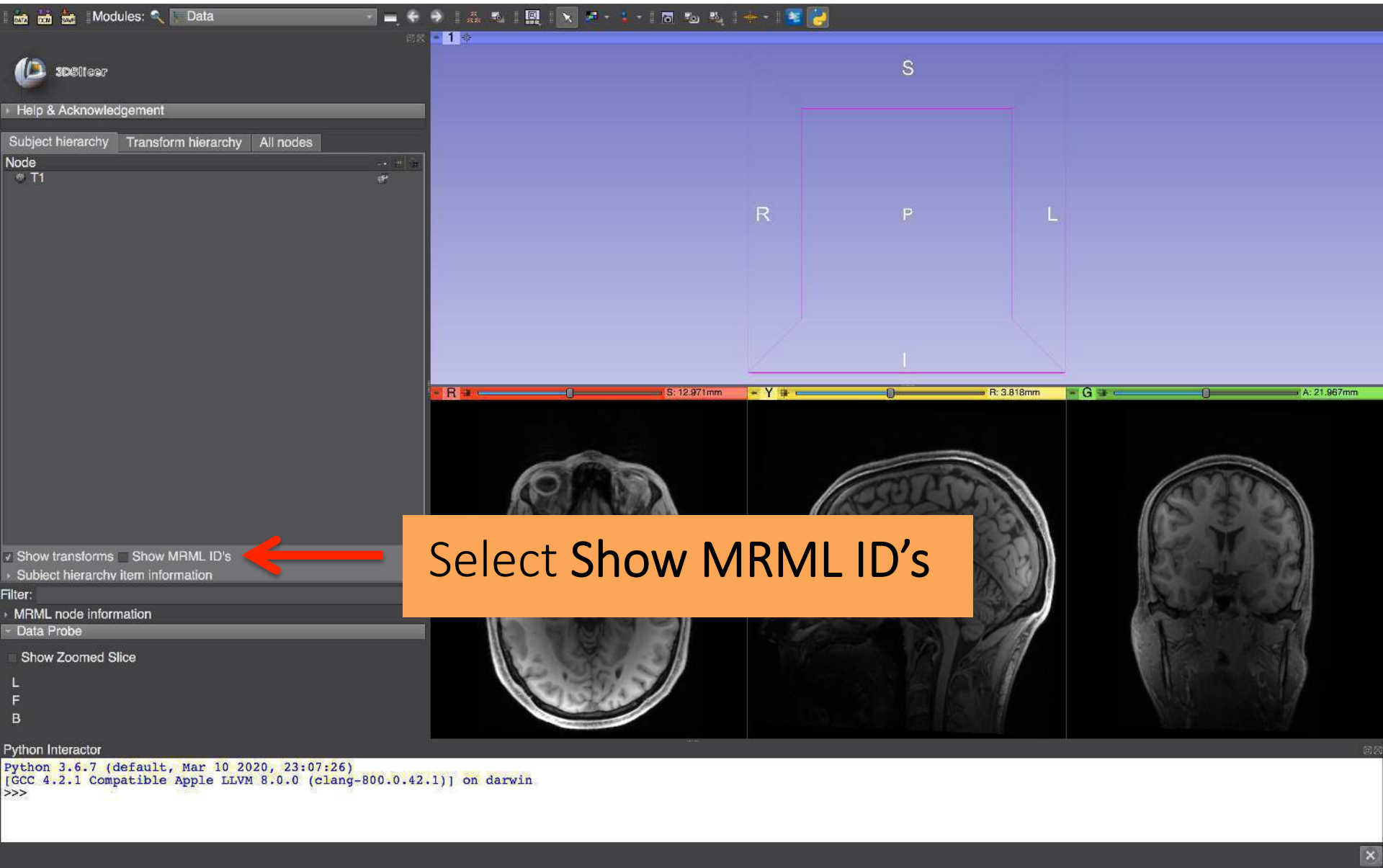


Display Node: Describes how the data should be visualized

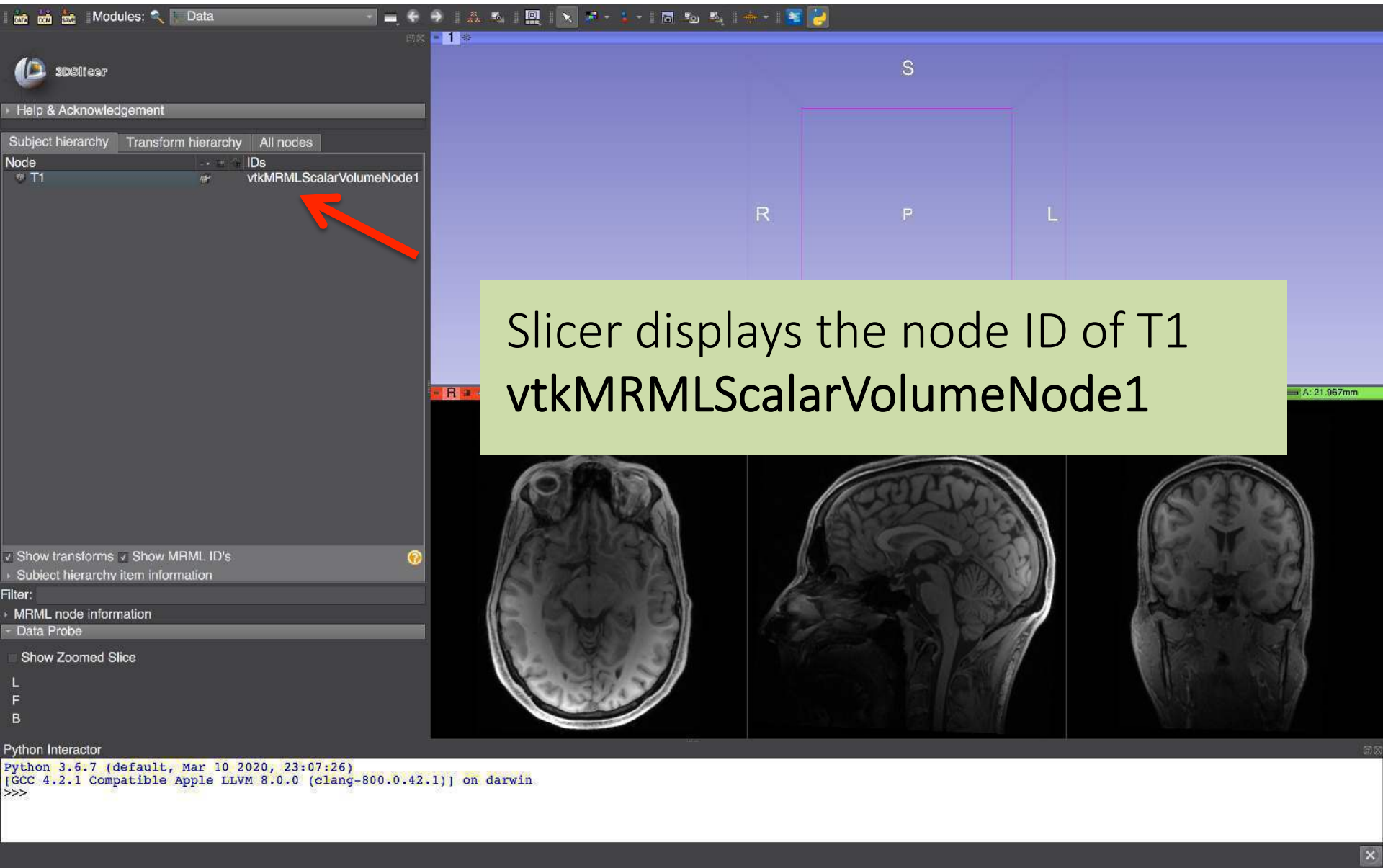


Storage Node: Describes how the data should be stored on disk

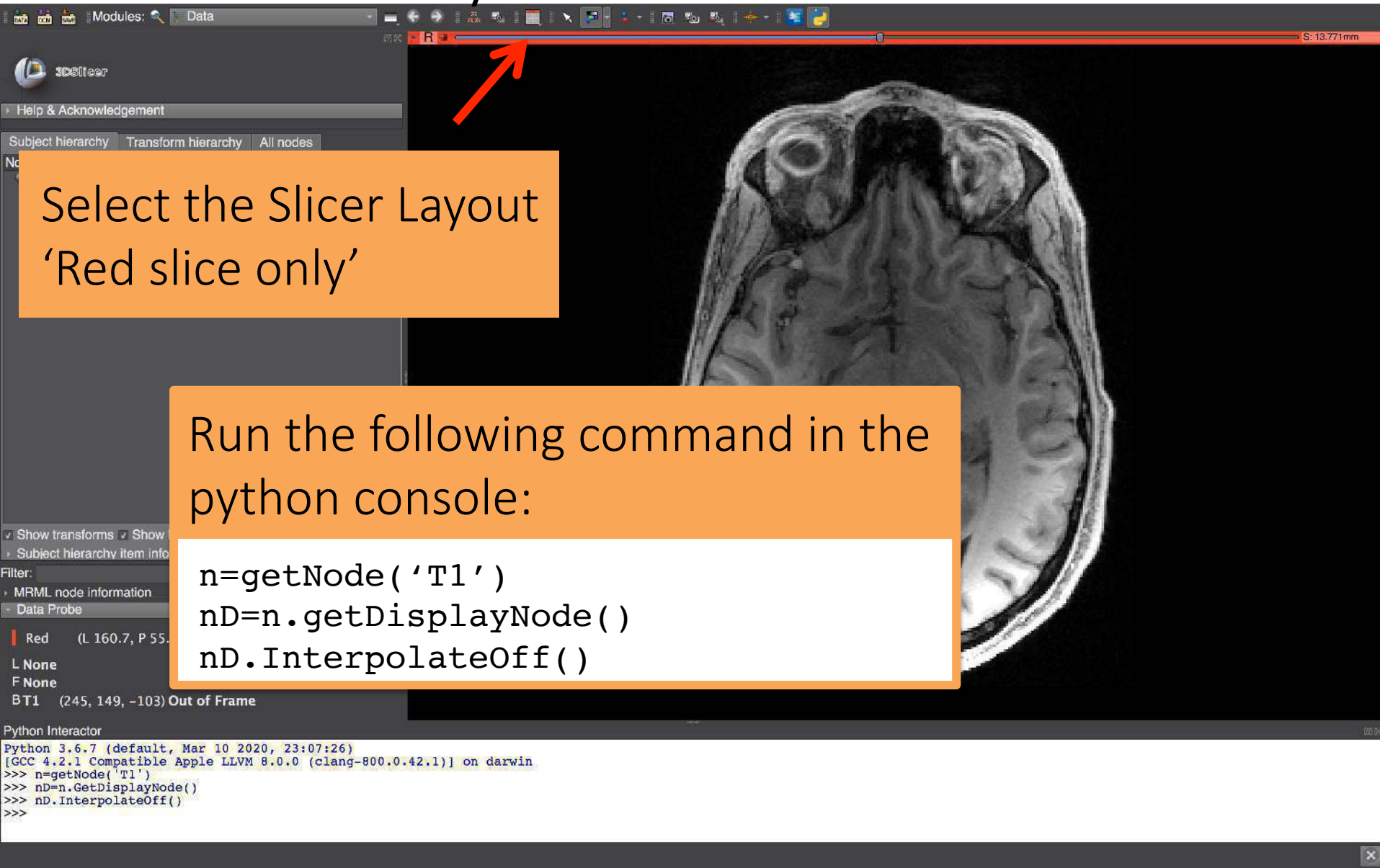
Tutorial dataset



Slicer Data Model



Accessing MRML nodes from the Python interactor



Select the Slicer Layout 'Red slice only'

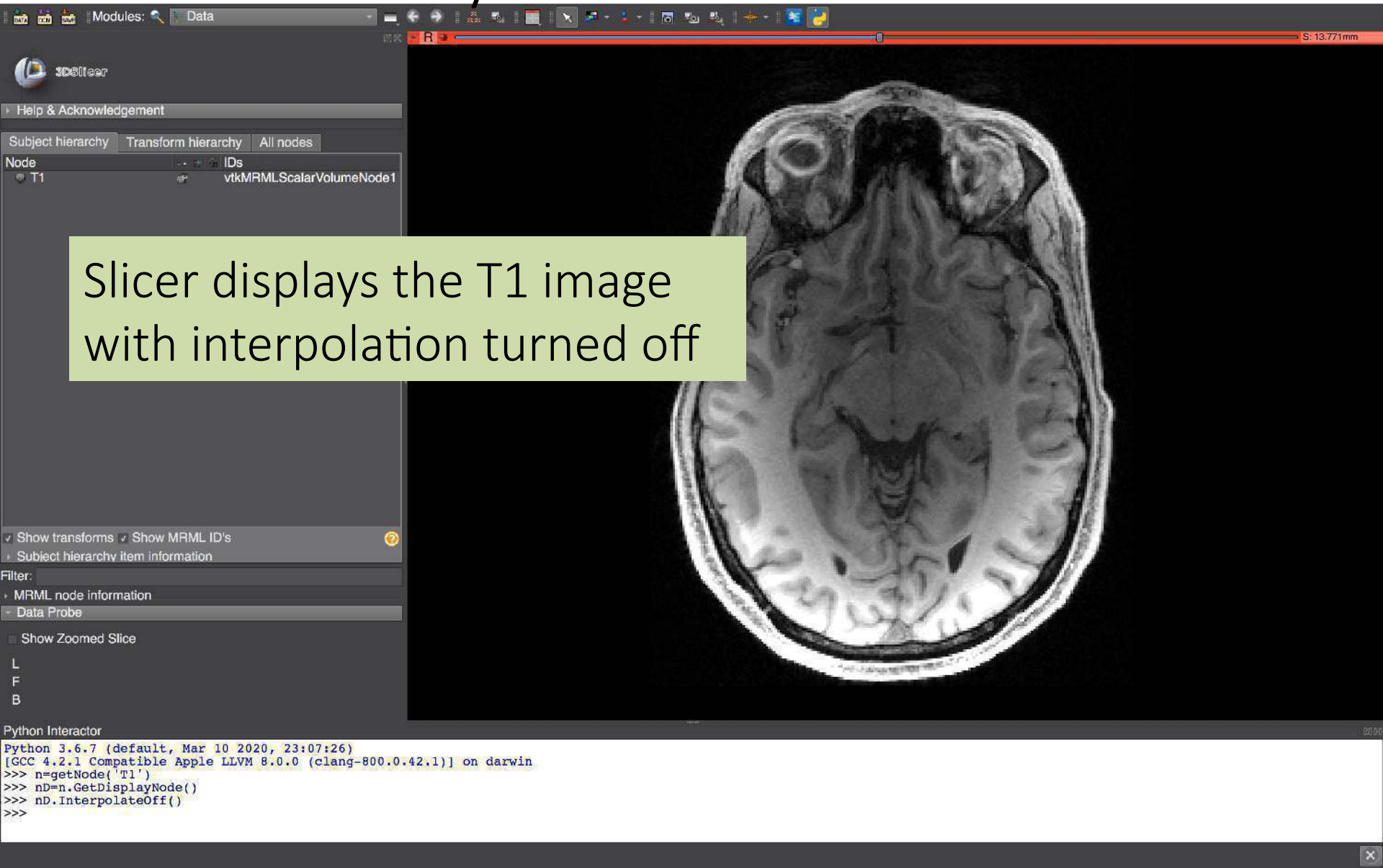
Run the following command in the python console:

```
n=getNode('T1')
nD=n.GetDisplayNode()
nD.InterpolateOff()
```

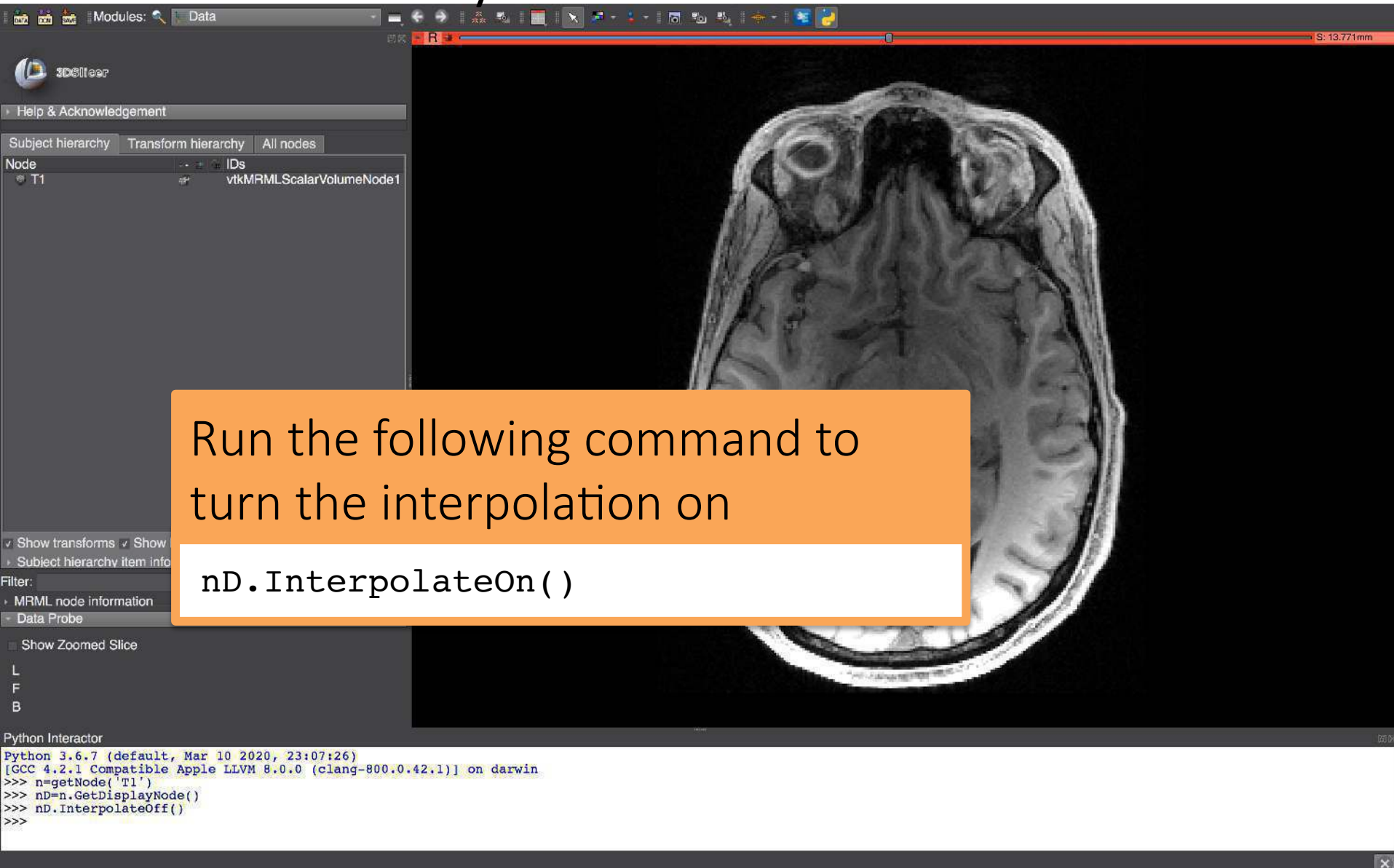
Python Interactor

```
Python 3.6.7 (default, Mar 10 2020, 23:07:26)
[GCC 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)] on darwin
>>> n=getNode('T1')
>>> nD=n.GetDisplayNode()
>>> nD.InterpolateOff()
>>>
```

Accessing MRML nodes from the Python interactor



Accessing MRML nodes from the Python interactor



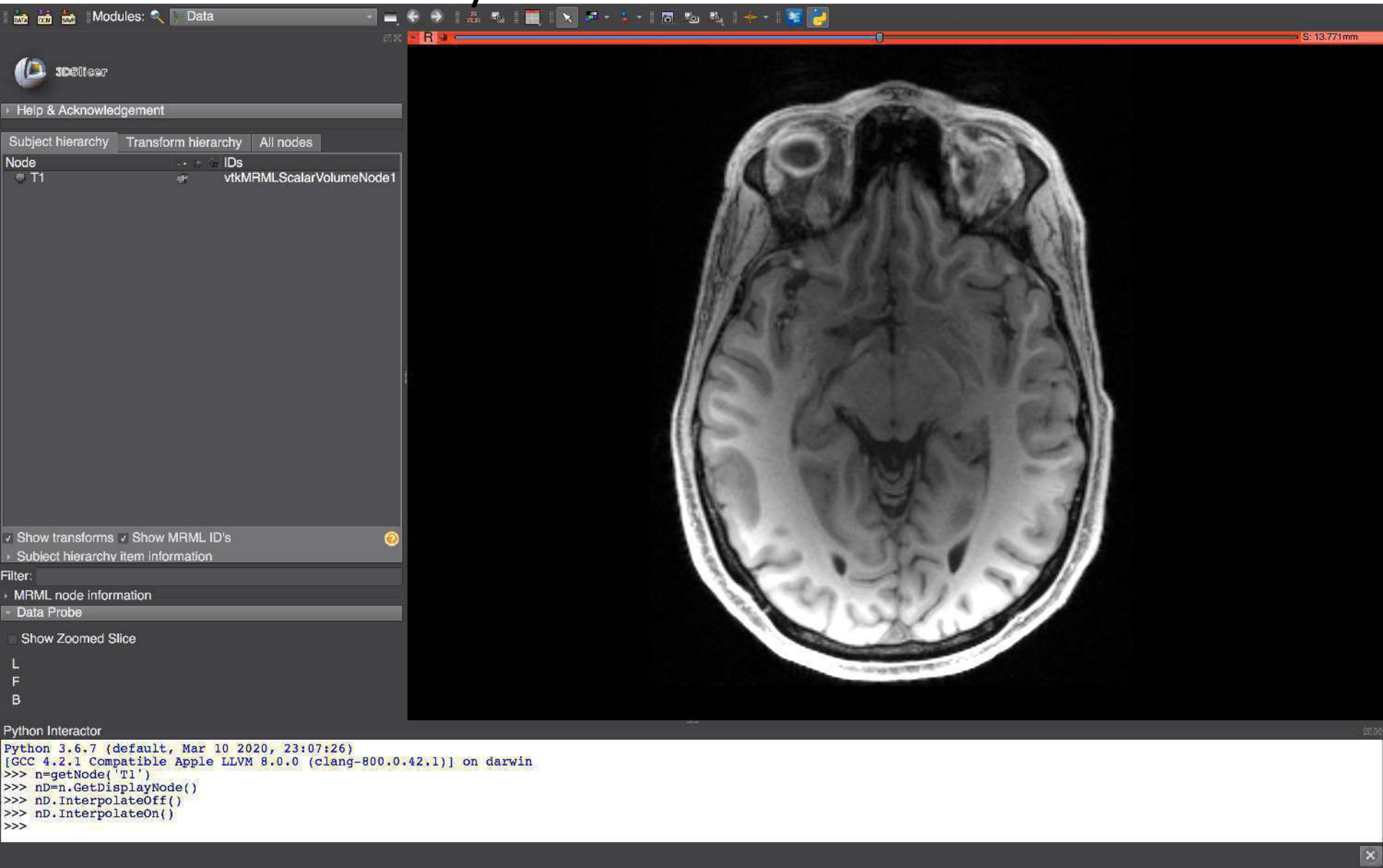
Run the following command to turn the interpolation on

```
nD.InterpolateOn()
```

Python Interactor

```
Python 3.6.7 (default, Mar 10 2020, 23:07:26)
[GCC 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)] on darwin
>>> n=getNode('T1')
>>> nD=n.GetDisplayNode()
>>> nD.InterpolateOff()
>>>
```

Accessing MRML nodes from the Python interactor



The screenshot displays the 3D Slicer software interface. The main window shows an axial MRI brain scan. The left sidebar contains the 'Subject hierarchy' panel, which lists the loaded data. Below this, there are checkboxes for 'Show transforms', 'Show MRML ID's', and 'Subject hierarchy item information'. The 'Python Interactor' panel at the bottom shows a series of commands and their output.

Subject hierarchy: Transform hierarchy All nodes

Node: T1 IDs: vtkMRMLScalarVolumeNode1

✓ Show transforms ✓ Show MRML ID's

Subject hierarchy item information

Filter:

MRML node information

Data Probe

Show Zoomed Slice

L
F
B

Python Interactor

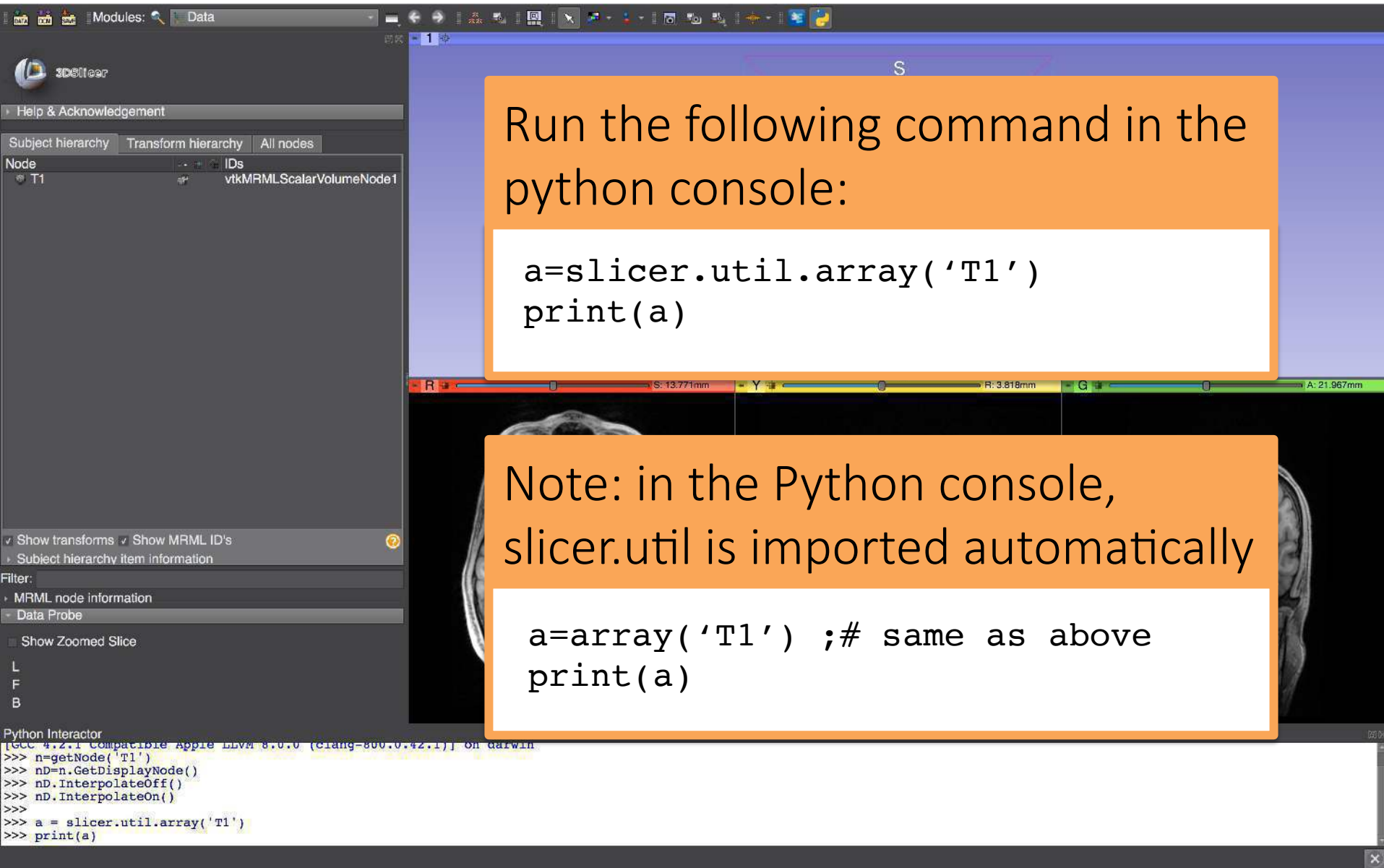
```
Python 3.6.7 (default, Mar 10 2020, 23:07:26)
[GCC 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)] on darwin
>>> n=getNode('T1')
>>> nd=n.GetDisplayNode()
>>> nd.InterpolateOff()
>>> nd.InterpolateOn()
>>>
```

Accessing voxels in a volume

- The `slicer.util` package gives access to volumes as NumPy multidimensional **arrays**
- Volumes can be modified using standard NumPy methods



Accessing voxels in a volume



Run the following command in the python console:

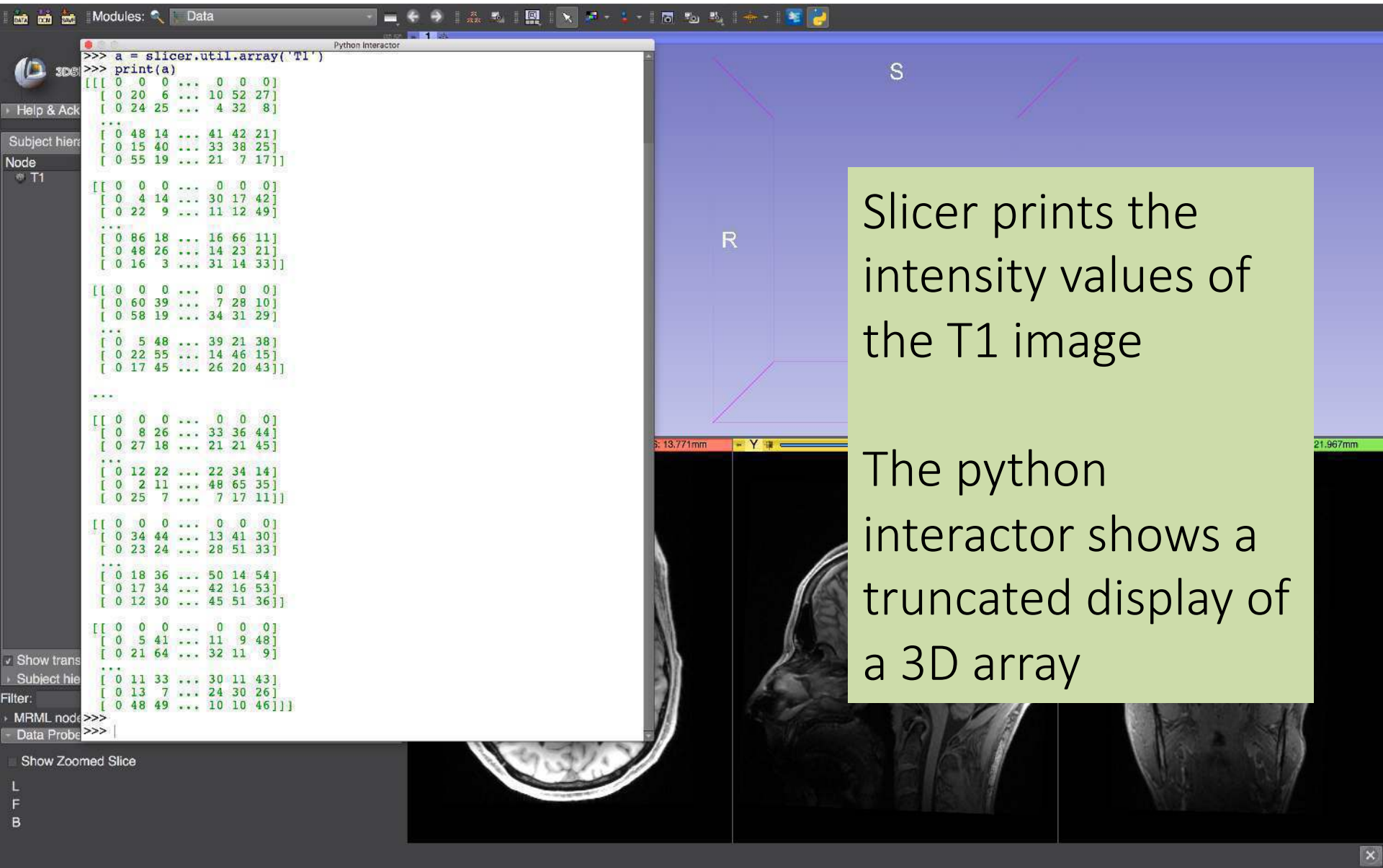
```
a=slicer.util.array('T1')  
print(a)
```

Note: in the Python console, `slicer.util` is imported automatically

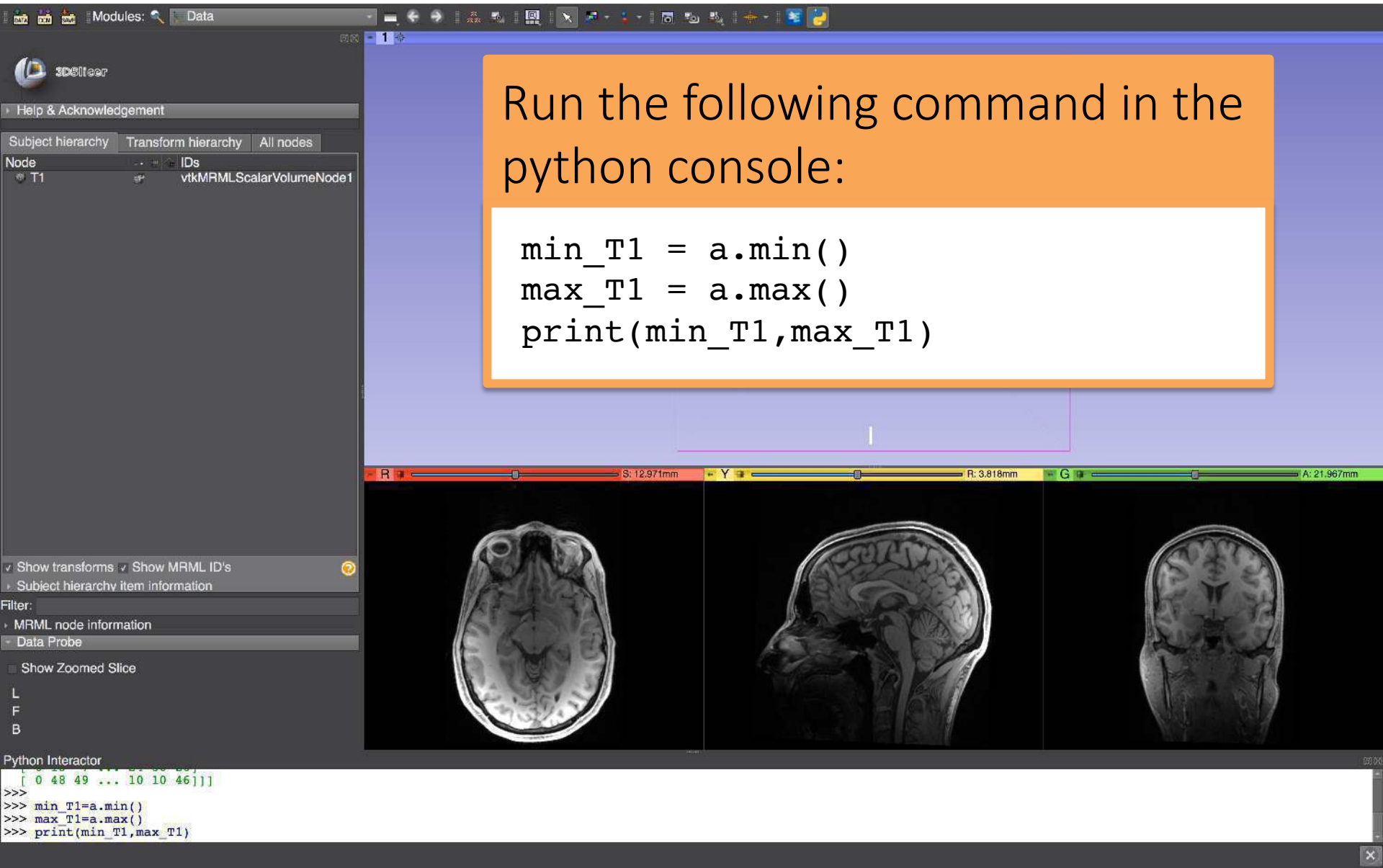
```
a=array('T1') ;# same as above  
print(a)
```

```
>>> n=getNode('T1')  
>>> nd=n.GetDisplayNode()  
>>> nd.InterpolateOff()  
>>> nd.InterpolateOn()  
>>>  
>>> a = slicer.util.array('T1')  
>>> print(a)
```

Accessing voxels in a volume



Accessing voxels in a volume

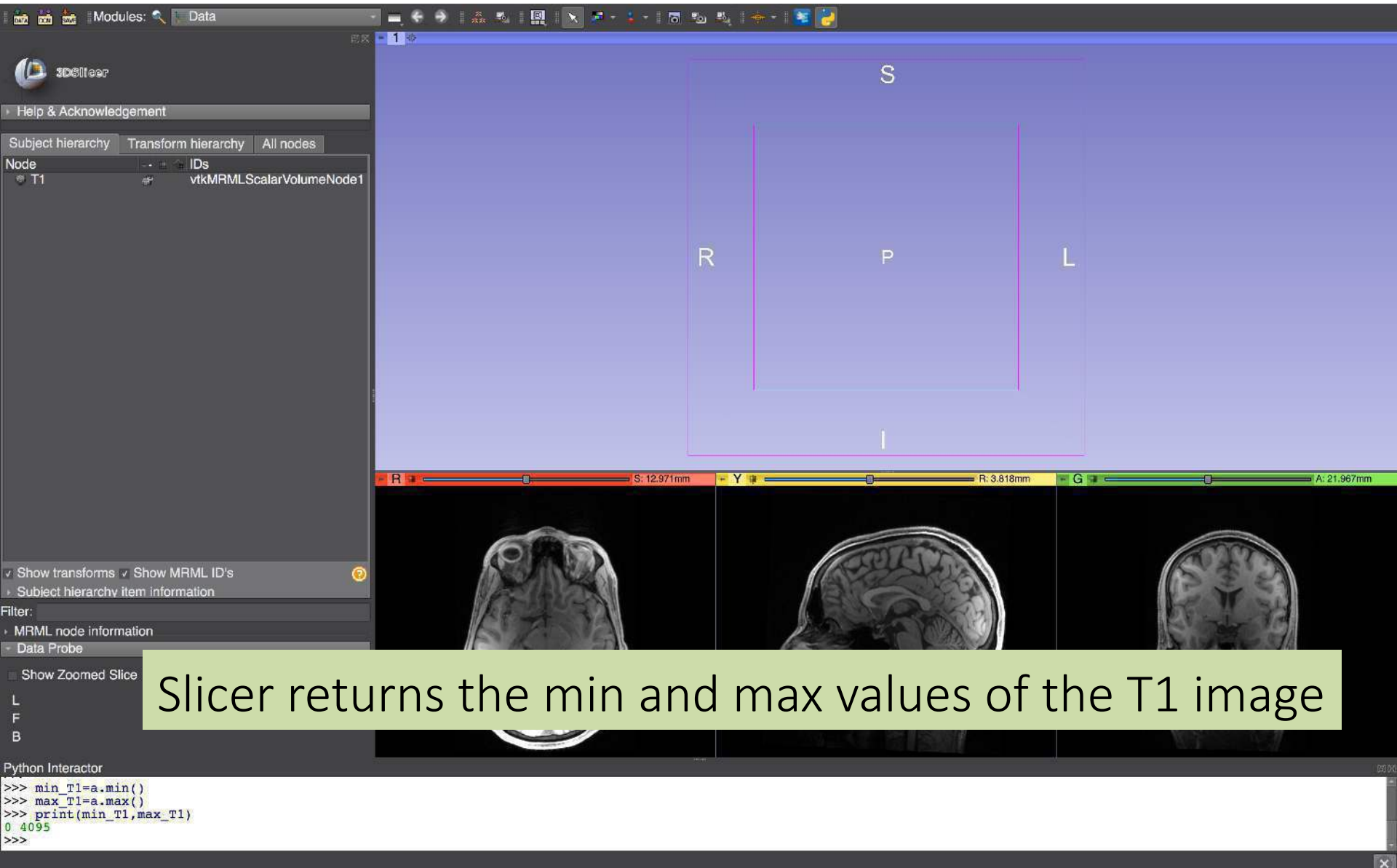


Run the following command in the python console:

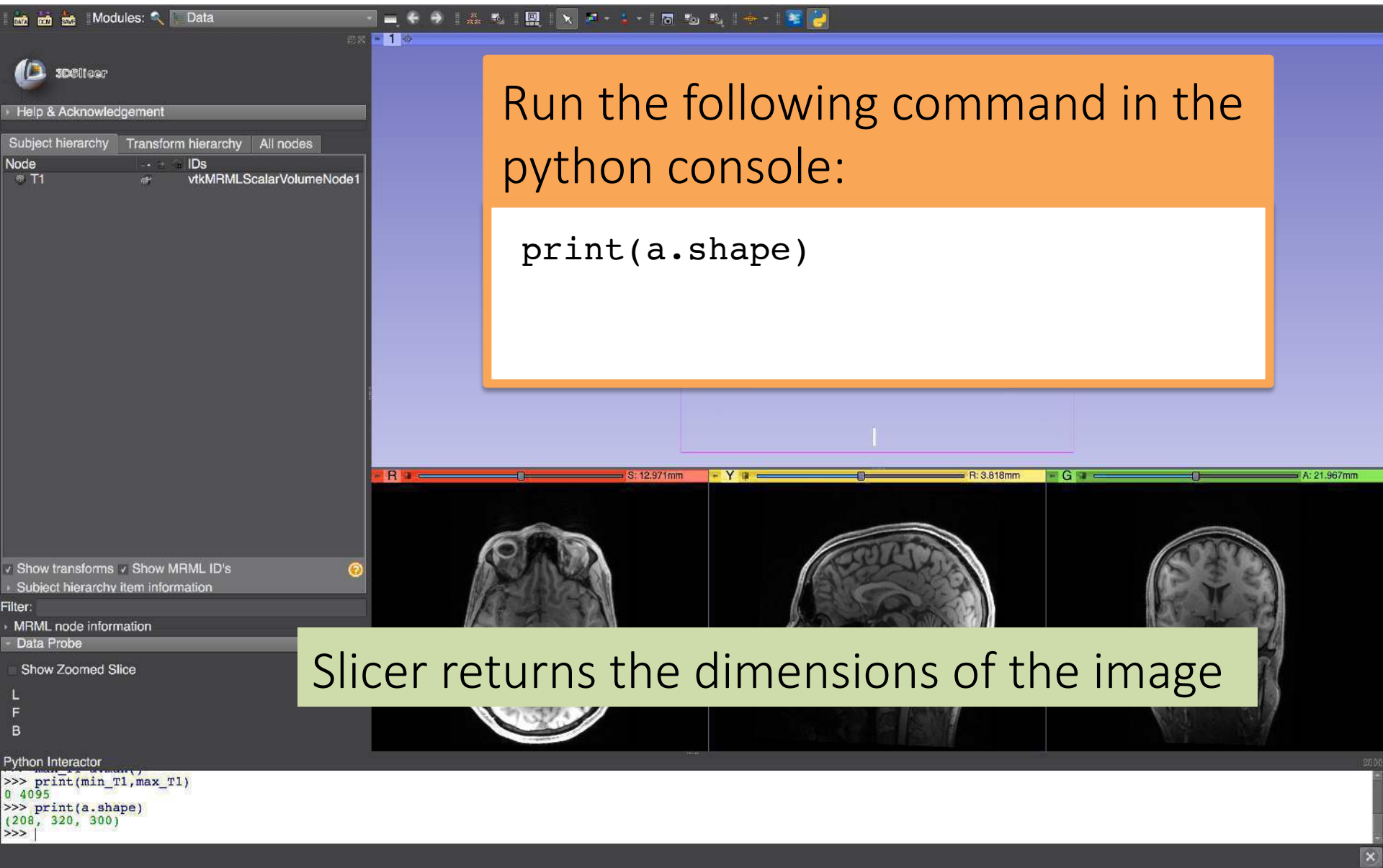
```
min_T1 = a.min()  
max_T1 = a.max()  
print(min_T1,max_T1)
```

The screenshot shows the 3D Slicer software interface. The left sidebar contains the 'Subject hierarchy' panel with a tree view showing 'T1' and 'vtkMRMLScalarVolumeNode1'. Below it are checkboxes for 'Show transforms', 'Show MRML ID's', and 'Subject hierarchy item information'. The bottom left corner features a 'Python Interactor' window with a list of data points and the execution of the commands shown in the orange box. The main 3D view displays three orthogonal slices of a brain MRI volume: axial (left), sagittal (middle), and coronal (right). Above the slices are sliders for the 'R' (Right) axis, 'Y' (Anterior-Posterior) axis, and 'G' (Superior-Inferior) axis, with numerical values indicating the current slice position.

Accessing voxels in a volume



Modifying voxels in a volume



The screenshot shows the 3D Slicer software interface. On the left, the 'Subject hierarchy' panel shows a node 'T1' of type 'vtkMRMLScalarVolumeNode1'. Below it, the 'Python Interactor' console shows the following code and output:

```
>>> print(min_T1,max_T1)
0 4095
>>> print(a.shape)
(208, 320, 300)
>>>
```

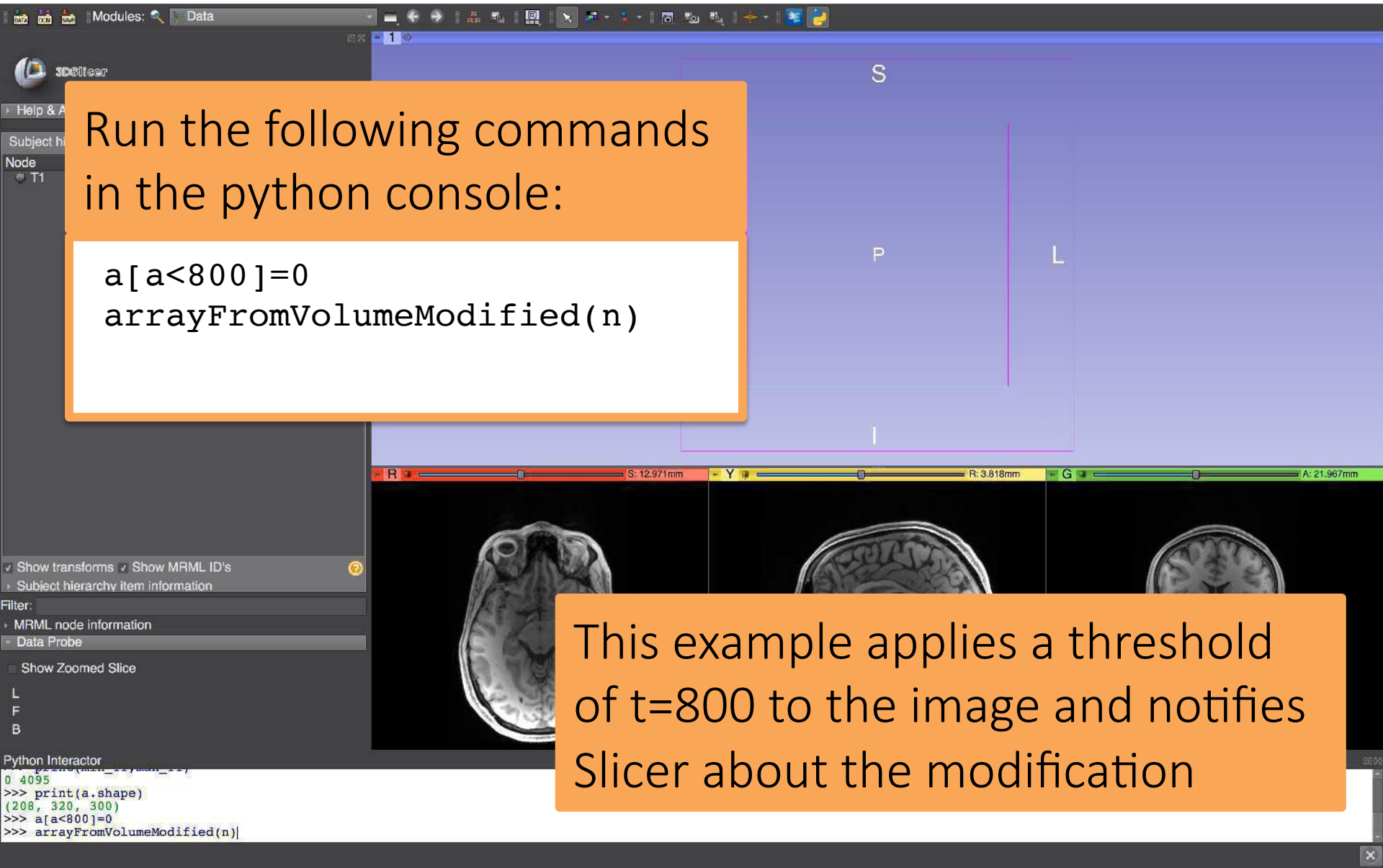
On the right, an orange box contains the text: 'Run the following command in the python console: print(a.shape)'. Below this, a green box contains the text: 'Slicer returns the dimensions of the image'. The main window displays three orthogonal views of a brain MRI volume (axial, sagittal, and coronal) with sliders for position and orientation.

Modifying voxels in a volume

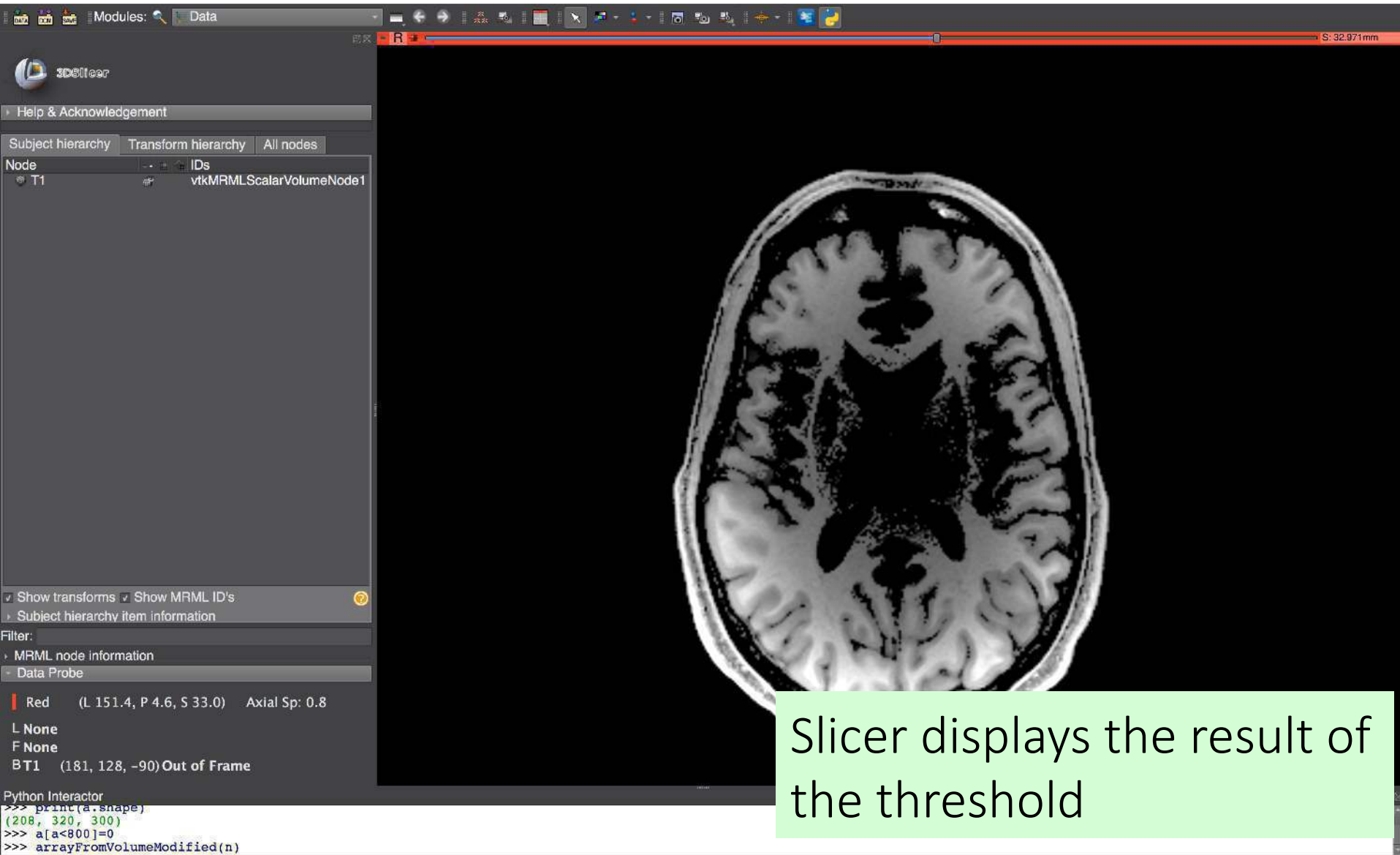
Run the following commands in the python console:

```
a[a<800]=0  
arrayFromVolumeModified(n)
```

This example applies a threshold of $t=800$ to the image and notifies Slicer about the modification



Modifying voxels in a volume



3D Slicer

Help & Acknowledgement

Subject hierarchy Transform hierarchy All nodes

Node IDs

T1 vtkMRMLScalarVolumeNode1

☒ Show transforms ☒ Show MRML ID's

Subject hierarchy item information

Filter:

MRML node information

Data Probe

Red (L 151.4, P 4.6, S 33.0) Axial Sp: 0.8

L None

F None

BT1 (181, 128, -90) Out of Frame

Python Interactor

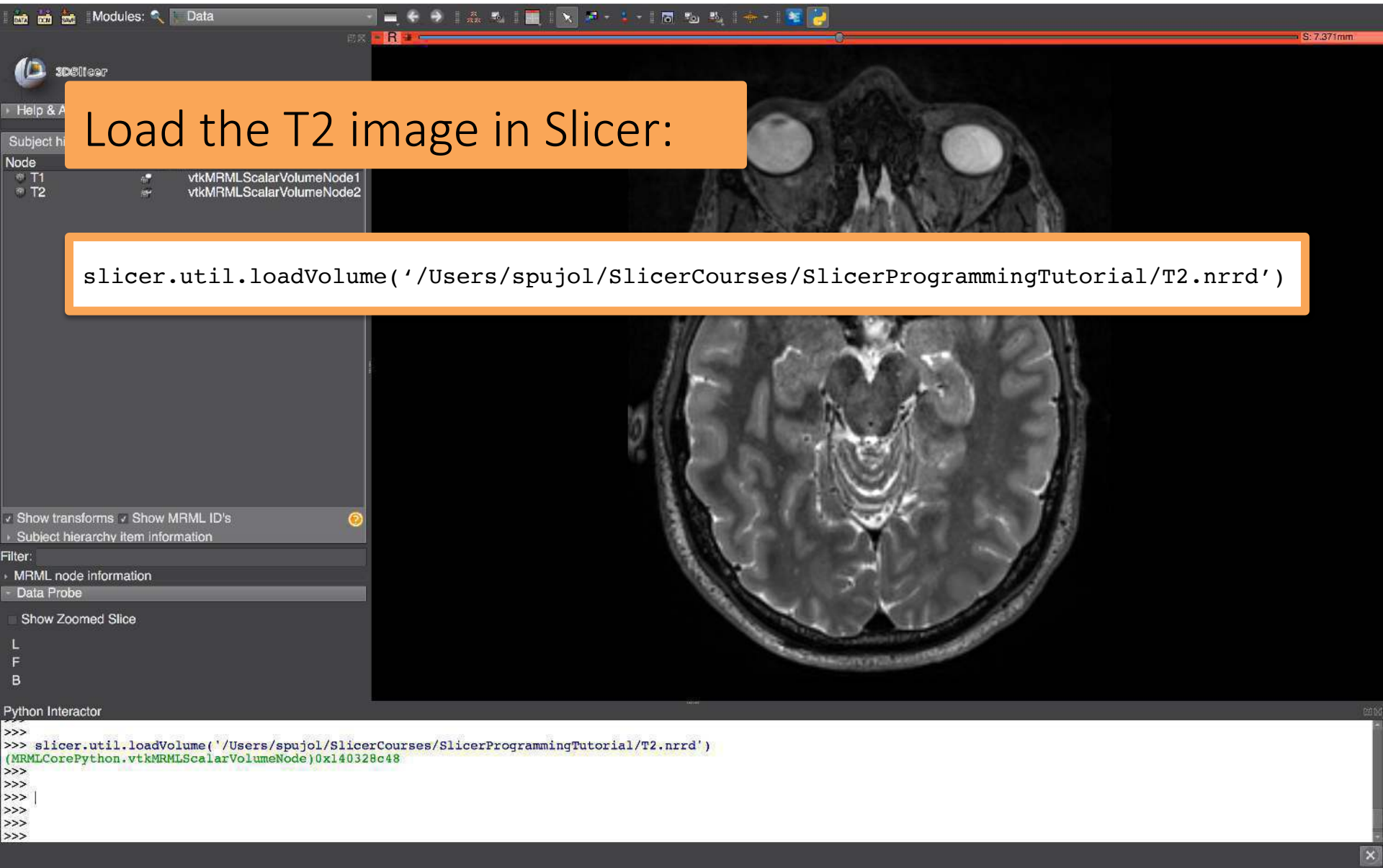
```
>>> print(a.shape)
(208, 320, 300)
>>> a[a<800]=0
>>> arrayFromVolumeModified(n)
```

Slicer displays the result of the threshold

Loading the T2 volume

Load the T2 image in Slicer:

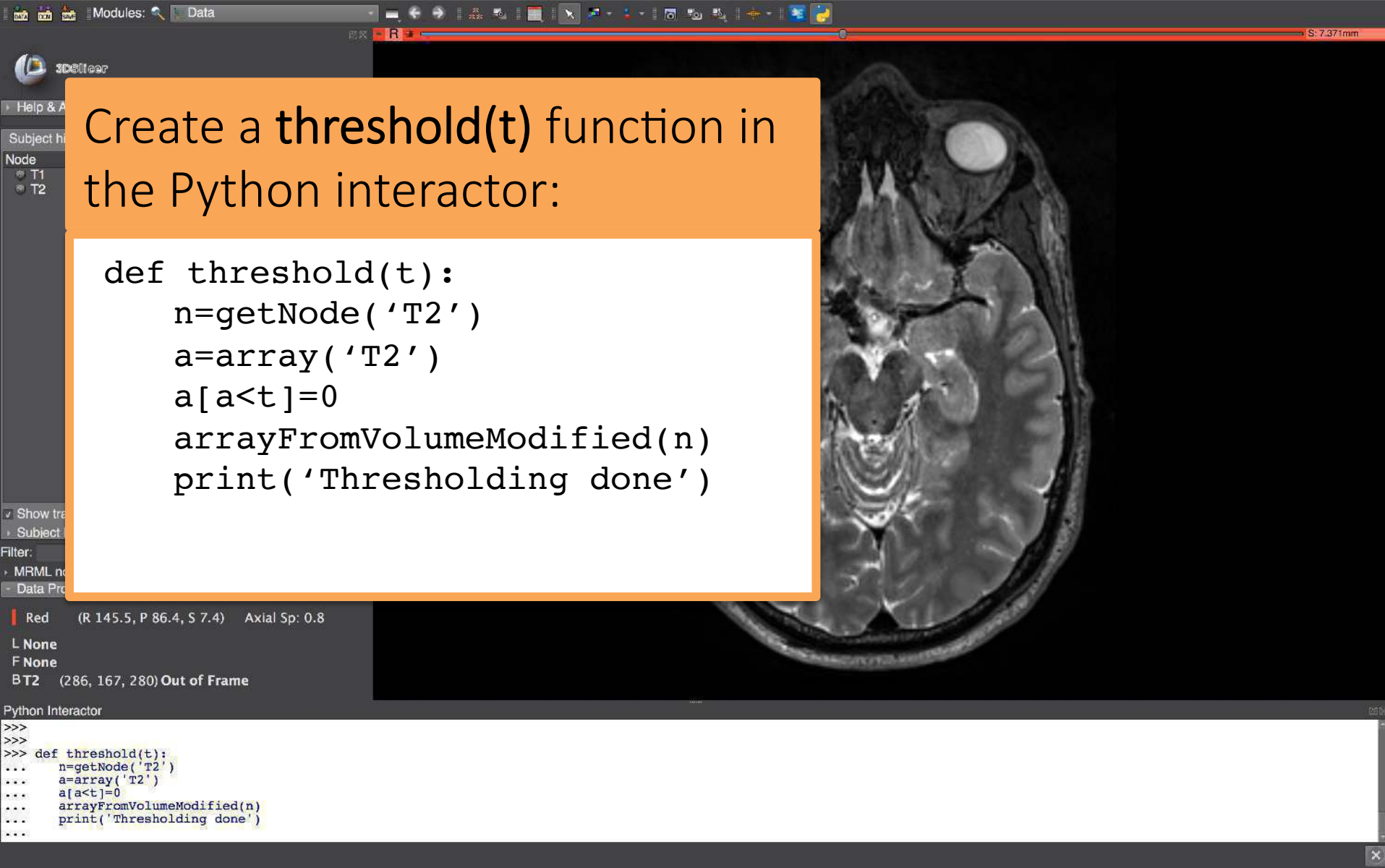
```
slicer.util.loadVolume('/Users/spujol/SlicerCourses/SlicerProgrammingTutorial/T2.nrrd')
```



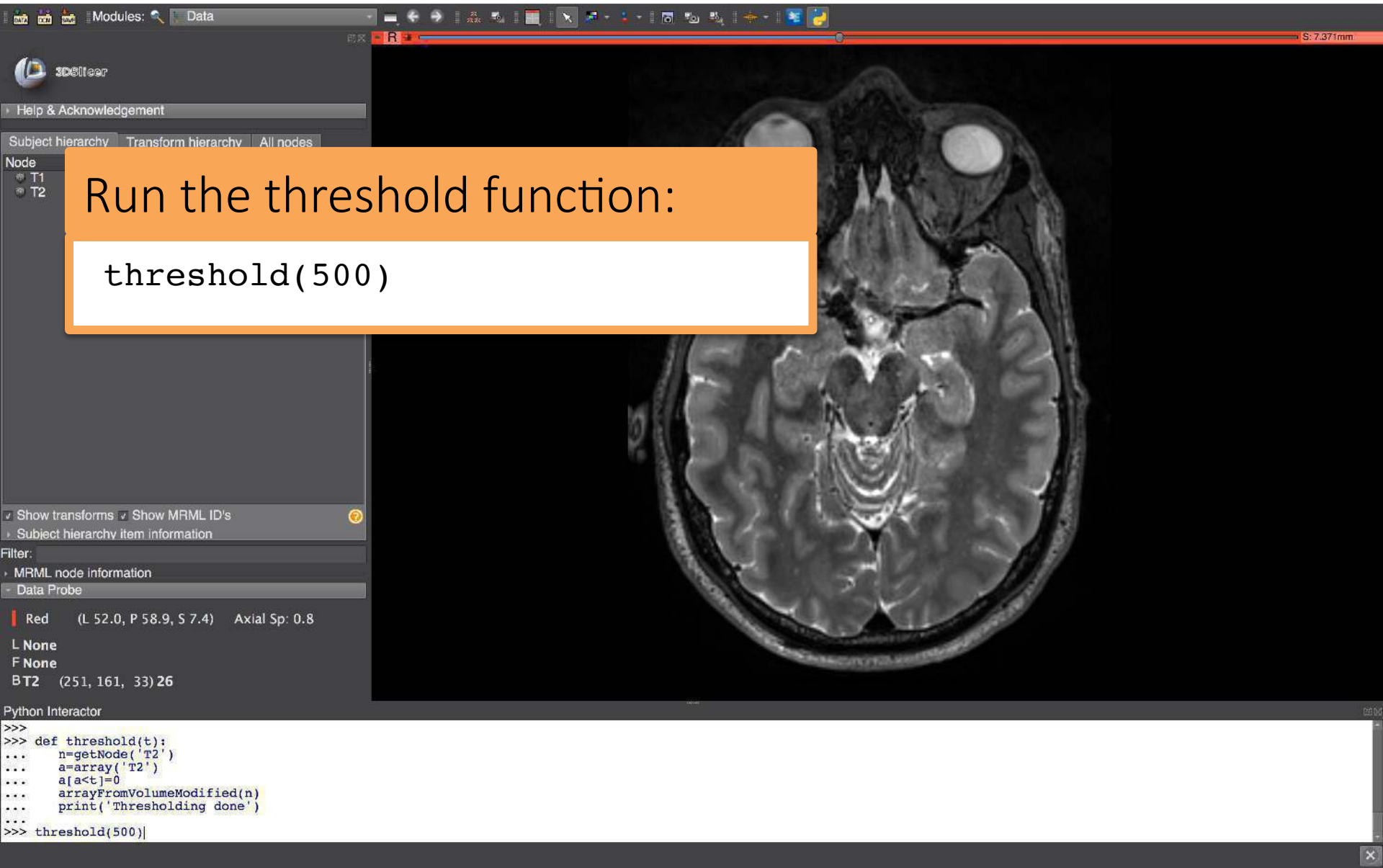
Python function: threshold

Create a **threshold(t)** function in the Python interactor:

```
def threshold(t):  
    n=getNode('T2')  
    a=array('T2')  
    a[a<t]=0  
    arrayFromVolumeModified(n)  
    print('Thresholding done')
```



Python function: threshold

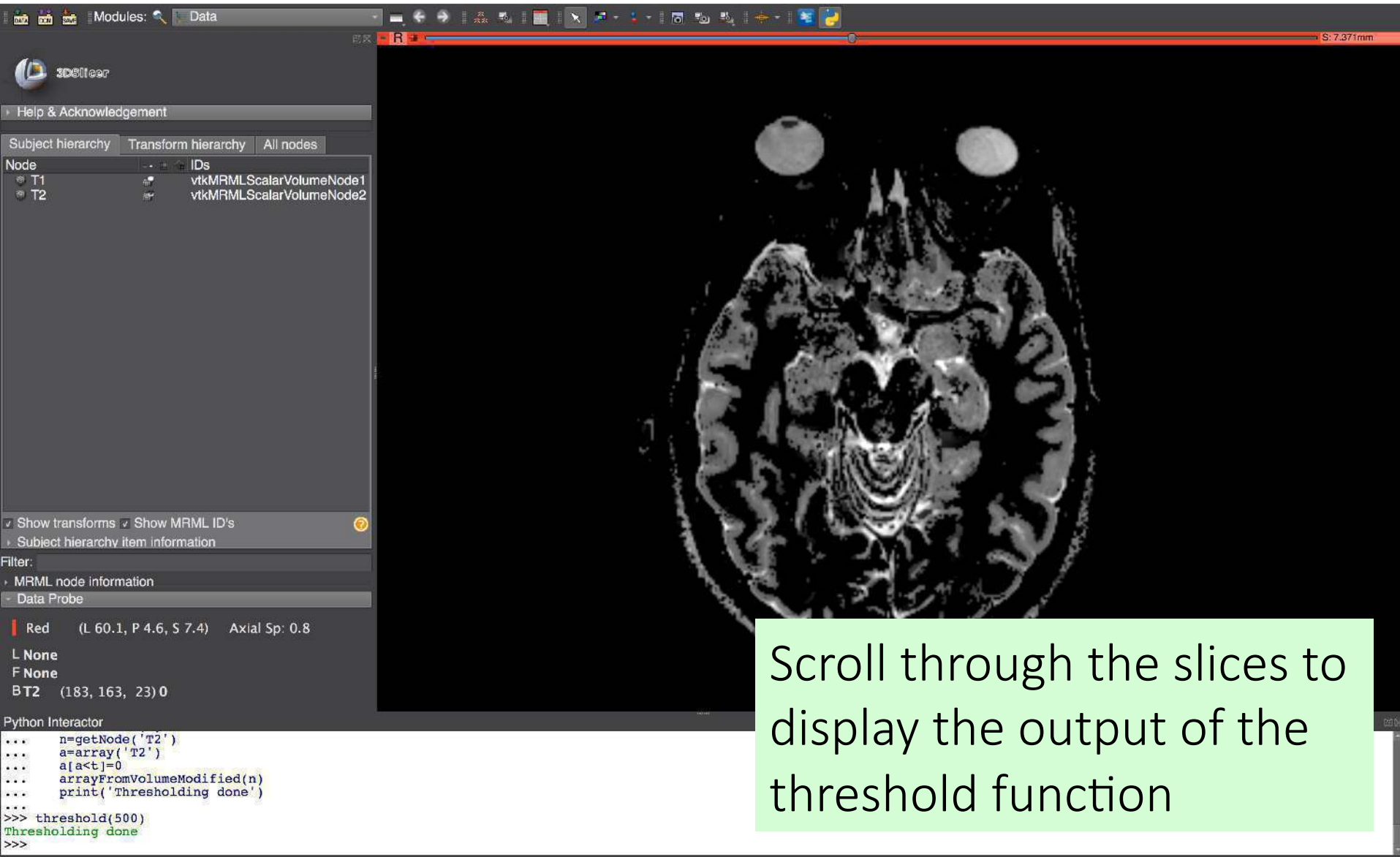


The screenshot displays the 3D Slicer software interface. The main window shows an axial brain MRI scan. An orange rectangular overlay is positioned over the left side of the brain, indicating a thresholded region. The left sidebar contains a 'Node' list with 'T1' and 'T2' entries. Below the node list, there are checkboxes for 'Show transforms', 'Show MRML ID's', and 'Subject hierarchy item information'. A 'Filter:' section is also present, with options for 'MRML node information' and 'Data Probe'. At the bottom left, a 'Python Interactor' window shows the following code:

```
>>>
>>> def threshold(t):
...     n=getNode('T2')
...     a=array('T2')
...     a[a<t]=0
...     arrayFromVolumeModified(n)
...     print('Thresholding done')
...
>>> threshold(500)
```

The bottom status bar displays the following information: 'Red (L 52.0, P 58.9, S 7.4) Axial Sp: 0.8', 'L None', 'F None', and 'BT2 (251, 161, 33) 26'.

Python function: threshold



The screenshot displays the 3D Slicer software interface. The main window shows an axial brain MRI slice. The left sidebar contains a 'Subject hierarchy' panel with nodes 'T1' and 'T2'. Below it, a 'Filter' panel shows 'MRML node information' and 'Data Probe'. At the bottom left, a 'Python Interactor' panel shows the following code:

```
>>> n=getNode('T2')
>>> a=array('T2')
>>> a[a<t]=0
>>> arrayFromVolumeModified(n)
>>> print('Thresholding done')
>>>
>>> threshold(500)
Thresholding done
>>>
```

A green text box on the right side of the image contains the text: "Scroll through the slices to display the output of the threshold function".

Big Picture

- Slicer provides easy access to analyze and modify complex data types
- Slicer is compatible with a wide range of Python scientific computing packages
- Slicer is a research environment for performing medical imaging experiments

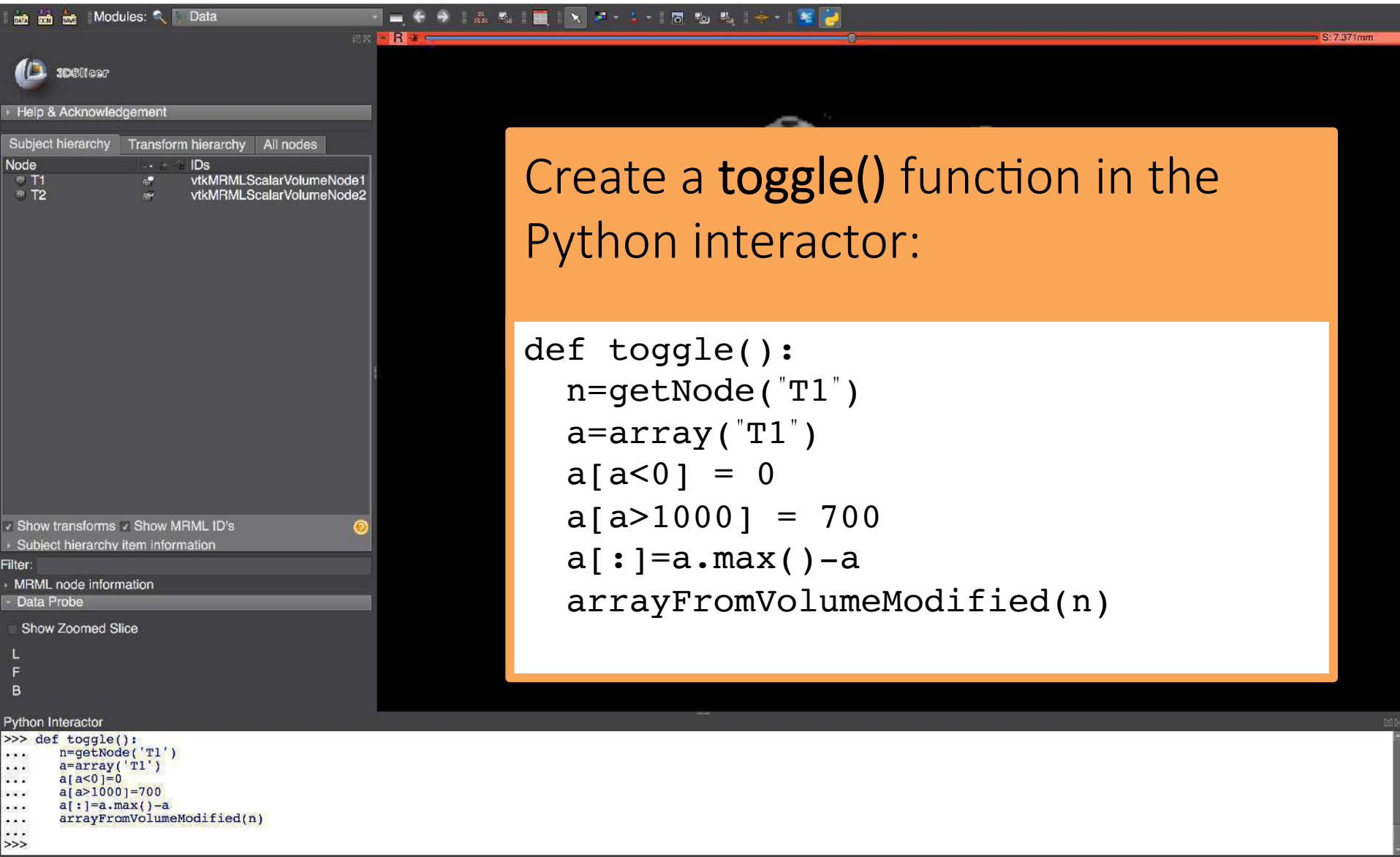
Part 3

Getting familiar with Qt in Slicer

Qt & PythonQt

- **Qt** is the main tool in Slicer to create widgets, dialogs, text entries, etc.
- **PythonQt** exposes most Qt functionalities and is accessible through the Python interactor in Slicer
- User interfaces can be created on the fly for rapid prototyping and debugging

Python function: toggle

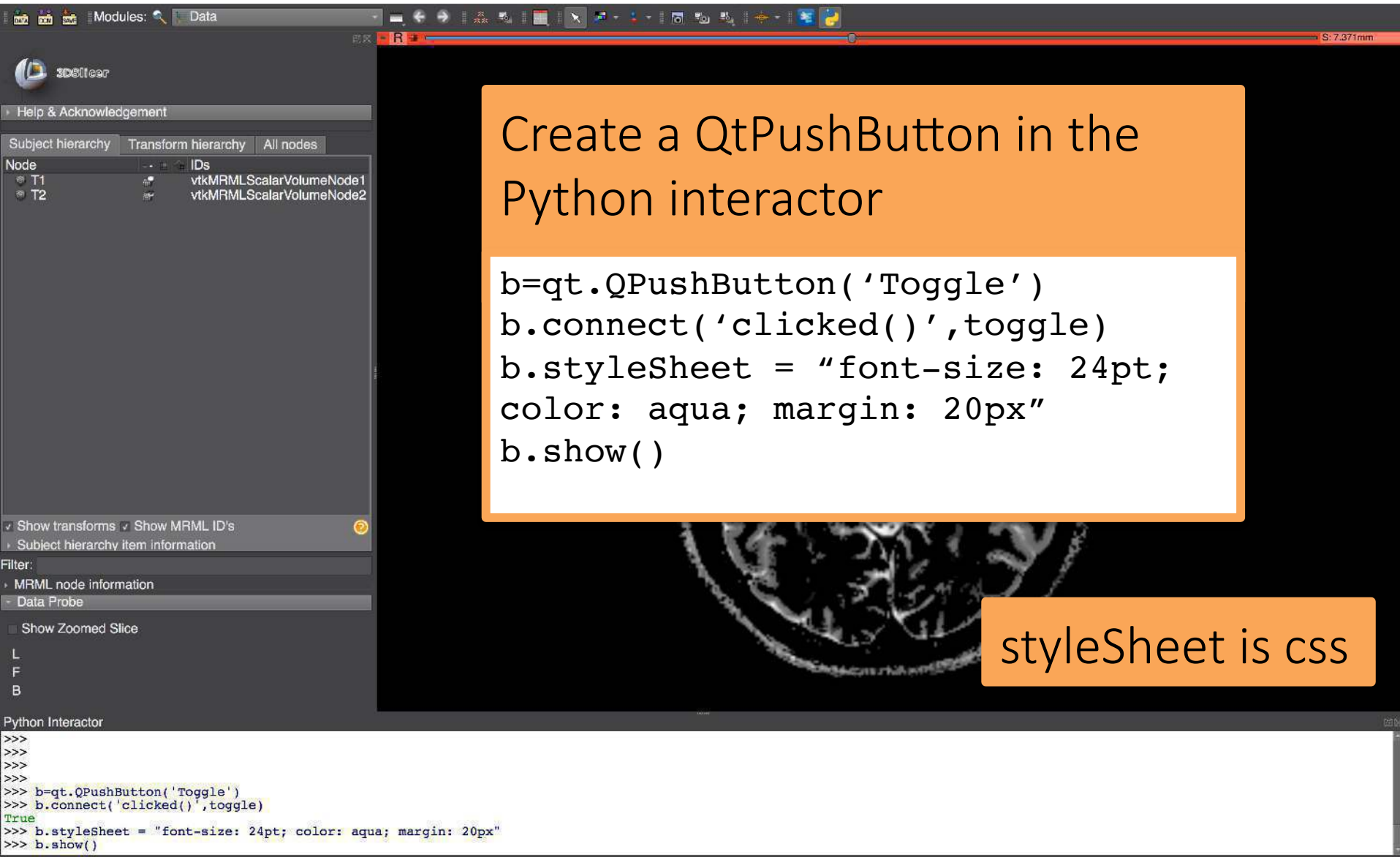


The screenshot shows the 3D Slicer software interface. On the left, the 'Node' list is visible under the 'Subject hierarchy' tab, showing two nodes: 'T1' (vtkMRMLScalarVolumeNode1) and 'T2' (vtkMRMLScalarVolumeNode2). Below the node list, there are checkboxes for 'Show transforms', 'Show MRML ID's', and 'Subject hierarchy item information'. At the bottom left, the 'Python Interactor' window is open, displaying the following code:

```
>>> def toggle():  
...     n=getNode('T1')  
...     a=array('T1')  
...     a[a<0]=0  
...     a[a>1000]=700  
...     a[:]=a.max()-a  
...     arrayFromVolumeModified(n)  
...  
>>>
```

On the right, an orange box contains the text: 'Create a toggle() function in the Python interactor:'.

Creating a Qt Push Button

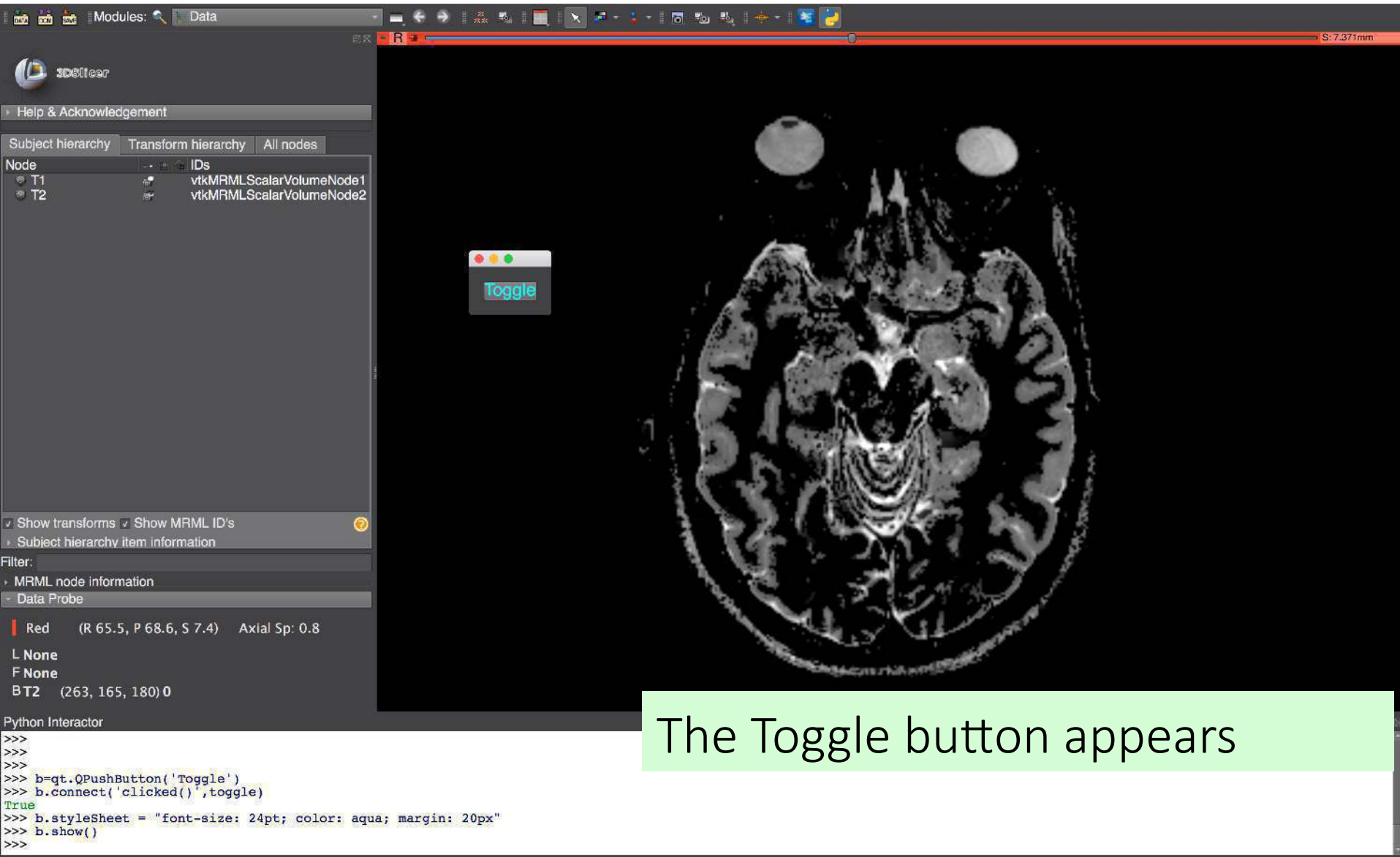


The screenshot shows the 3D Slicer software interface. On the left, the 'Subject hierarchy' panel lists two nodes: T1 (vtkMRMLScalarVolumeNode1) and T2 (vtkMRMLScalarVolumeNode2). The 'Python Interactor' at the bottom contains the following code:

```
>>>
>>>
>>>
>>>
>>> b=qt.QPushButton('Toggle')
>>> b.connect('clicked()',toggle)
True
>>> b.styleSheet = "font-size: 24pt; color: aqua; margin: 20px"
>>> b.show()
```

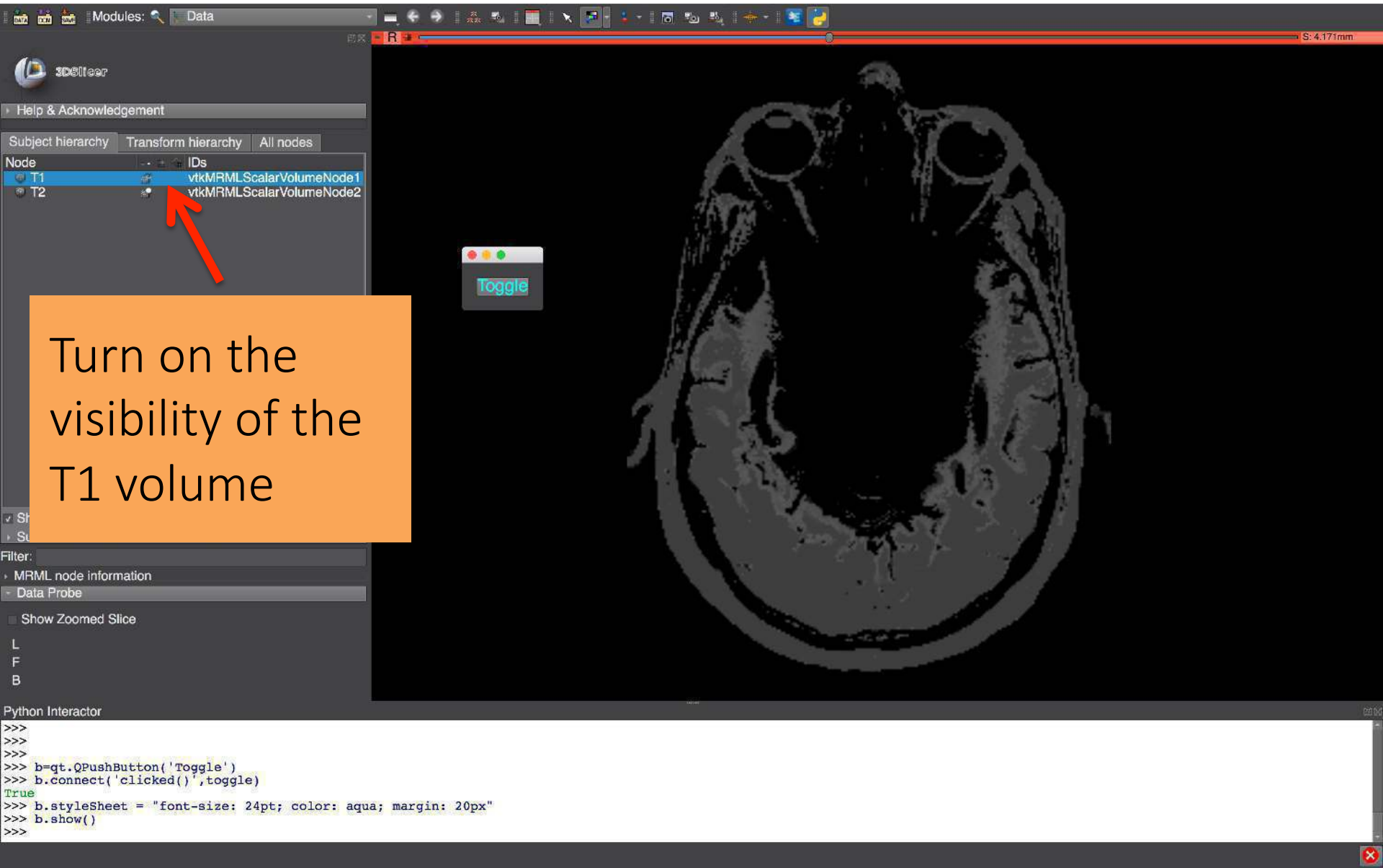
On the right, an orange box contains the text: "Create a QPushButton in the Python interactor". Below this, another orange box contains the text: "styleSheet is css".

Creating a Qt Push Button



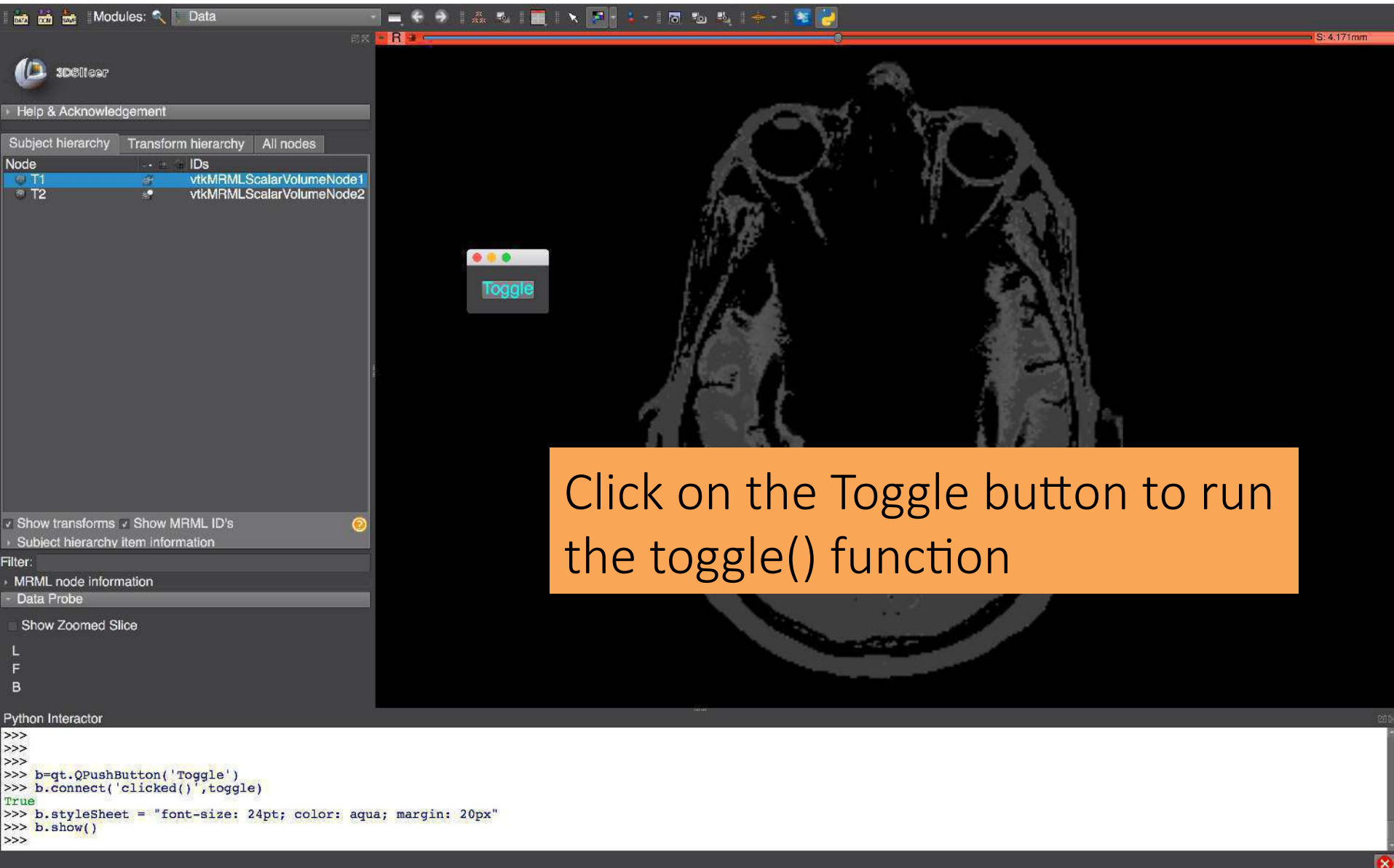
The Toggle button appears

Creating a Qt Push Button



Turn on the
visibility of the
T1 volume

Creating a Qt Push Button

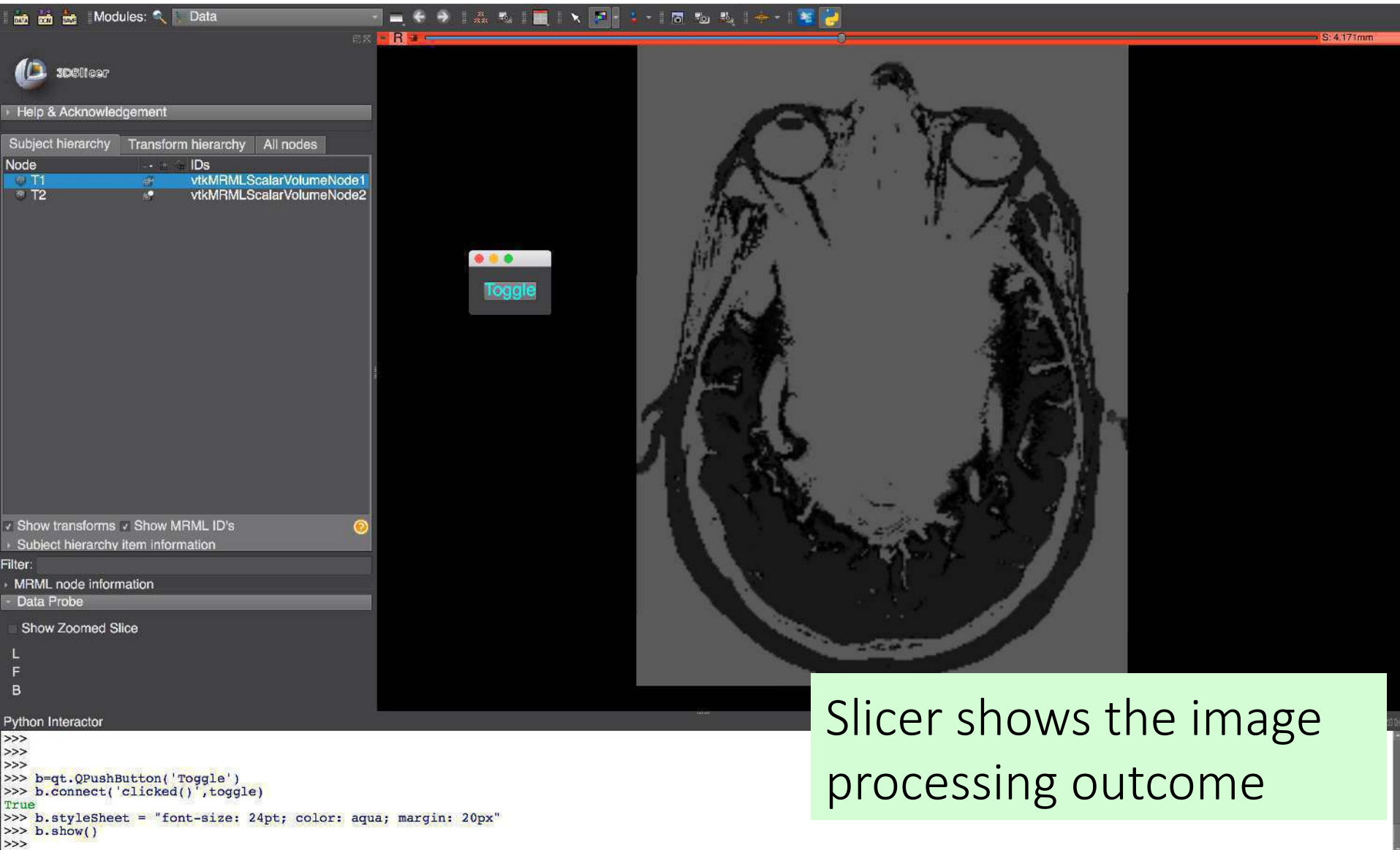


The screenshot displays the 3D Slicer application window. The main 3D view shows a grayscale axial CT scan of a human head. Overlaid on this view is a small Qt window with a title bar and a single button labeled "Toggle" in cyan text. The left sidebar contains several panels: "Help & Acknowledgement", "Subject hierarchy" (showing nodes T1 and T2), "Transform hierarchy", "All nodes", "Filter:" (with "MRML node information" and "Data Probe" selected), and "Show Zoomed Slice". The bottom panel is the "Python Interactor", which contains a Python script.

Click on the Toggle button to run the toggle() function

```
>>>
>>>
>>>
>>> b=qt.QPushButton('Toggle')
>>> b.connect('clicked()',toggle)
True
>>> b.setStyleSheet = "font-size: 24pt; color: aqua; margin: 20px"
>>> b.show()
>>>
```

Creating a Qt Push Button



Slicer shows the image processing outcome

Examples of scripted modules

- The tutorial demonstrates how to create a simple interface in Python
- Slicer integrates many sophisticated scripted modules such as Segment Statistics, Sample Data, Endoscopy module, etc.
- For further reading, please look at the Slicer Script Repository:

<https://www.slicer.org/wiki/Documentation/Nightly/ScriptRepository>

Conclusion

- Slicer enables developers to create complex interfaces that are streamlined for target users
- The software platform provides unlimited customization possibilities
- Slicer gives access to advanced underlying libraries through a cross-platform package that is easy to deploy to end-users

Acknowledgments



Neuroimage Analysis Center
(NIBIB P41 EB015902)



Sylvain Bouix, Ph.D.

Psychiatry Neuroimaging Laboratory